



Full Length Article

Bridging industrial control systems design and testing through threat modeling-driven penetration testing - a microgrid case study

Shaymaa Mamdouh Khalil ^{a,*}, Hayretudin Bahsi ^{a,b}, Tarmo Korõtko ^c

^a School of Information Technologies, Tallinn University of Technology, Estonia

^b School of Informatics, Computing, and Cyber Systems, Northern Arizona University, United States

^c School of Engineering, Tallinn University of Technology, Estonia

ARTICLE INFO

Keywords:

Threat modeling
Cyber-physical systems (CPS)
Industrial control systems (ICS)
Penetration testing
STRIDE
MITRE ATT&CK for ICS

ABSTRACT

Threat modeling is a widely recommended approach for supporting penetration test planning, while penetration testing can, in turn, verify and validate design-phase threat modeling outcomes. Yet, this interrelation remains largely overlooked in academic research. To address this gap, this study proposes BRIDGE, a nine-stage threat modeling-driven penetration testing methodology that connects the design and testing phases. Guided by design-phase threat modeling results, the methodology supports structured test selection, enabling a more focused and efficient testing process. The study introduces a mapping of MITRE ATT&CK for ICS techniques to STRIDE threat categories, establishing a key link between high-level threat analysis performed during system design and the intermediate-level attack representation required for penetration test planning. The methodology's practical applicability is demonstrated through a real-world case study of a recently deployed microgrid system. The study also examines the effectiveness of CVSS v4.0 compared to v3.1 in representing the distinctive risk profile of ICS vulnerabilities, considering both security requirements and potential safety impacts. This research provides practical guidance for ICS cybersecurity practitioners to enhance penetration test planning efficiency, ensure adequate coverage of critical threat testing, and streamline collaboration with third-party testers. Researchers can leverage the proposed methodology and the MITRE ATT&CK to STRIDE mapping to develop detailed ICS testing procedures, thereby contributing to the advancement of structured ICS security testing practices.

1. Introduction

Industrial Control Systems (ICS) are complex systems of systems that integrate Information Technology (IT) and Operational Technology (OT). Given their vital role in critical infrastructure, securing ICS against cyber attacks remains a persistent challenge (Stouffer et al., 2023). Since the discovery of Stuxnet in 2010, the adoption of secure-by-design principles has become essential in ICS development, with threat modeling playing a key role in the design phase. Despite growing research interest in ICS threat modeling, there remains a lack of readily deployable and empirically validated approaches tailored to ICS environments (Khalil et al., 2024). Moreover, validating threat modeling outcomes and ensuring that selected security requirements are effectively implemented remain significant challenges.

In parallel, ICS penetration testing presents distinct technical and organizational challenges. The integration of IT and OT components results in a broad and complex attack surface, requiring expertise across multiple domains. Unlike IT systems, ICS operate under strict availability and safety constraints, making thorough penetration test planning

essential to avoid service disruption or physical damage (Stouffer et al., 2023). Given the potential real-world consequences of ICS attacks, including risks to human safety, ICS penetration testing must consider not only technical expertise, but also the impact and consequences of each selected test.

Threat modeling-based penetration testing methodologies, where threat modeling supports penetration test planning, can help ensure more comprehensive testing and provide a clear rationale for test selection (Süren et al., 2023). Although several threat modeling-based penetration testing methodologies have been proposed for Cyber-Physical Systems (CPS) (see Section 2), they are primarily designed for Internet of Things (IoT) systems, leaving a significant gap in addressing the unique challenges of ICS. While IoT and ICS share common attack surfaces, they differ significantly in system complexity, safety constraints, and deployment context. Consequently, IoT-focused penetration testing methodologies do not adequately address the planning needs of ICS penetration testing.

Furthermore, existing CPS threat modeling-based penetration testing studies predominantly focus on laboratory environments or

* Corresponding author.

E-mail addresses: Shaymaa.Khalil@taltech.ee (S.M. Khalil), Hayretudin.Bahsi@taltech.ee (H. Bahsi), tarmo.korotko@taltech.ee (T. Korõtko).

controlled testbeds, which limits the consideration of real-world project constraints and deployment conditions (Süren et al., 2023; Salzillo et al., 2020), revealing a clear gap in real-world case studies that consider project-specific constraints and practical deployment conditions. This gap highlights the need for methodologies that address the complexities of real ICS projects and connect the various development stages, such as incorporating design-phase threat modeling insights to support penetration test planning, as demonstrated in this study.

Resource and time limitations may necessitate various prioritization decisions in the penetration testing process for resource-constrained projects that involve the development of new system components, such as the microgrid system described by Khalil et al. (2023), where a control system was developed. In such contexts, stakeholders might decide to exclude one or more of the four International Electrotechnical Commission (IEC) 62443 component security testing processes (see Section 2) for the newly developed components. Consequently, penetration testing becomes particularly critical, as it indirectly evaluates the effectiveness of the implemented threat mitigation techniques by attempting to exploit the system's vulnerabilities.

A previous study (Khalil et al., 2023) introduced a CPS threat modeling methodology intended for application during the early stage of the design phase, when detailed information about the system software and vendors is typically unavailable. For this reason, a high-level threat modeling method, STRIDE (Howard and Lipner, 2006), was adopted to analyze potential system threats at a high level of abstraction. In the first case study (Khalil et al., 2023), STRIDE proved effective in establishing a clear, understandable common language for cybersecurity specialists and non-cybersecurity experts (e.g., software and power engineers) involved in the threat modeling exercise. The choice of a non-sophisticated threat modeling method, similar to STRIDE, during the design phase facilitated brainstorming and discussions about threat impacts and consequences. However, following system deployment and during planning of the testing phase, discussions of test scenarios require a more detailed framework, such as MITRE ATT&CK for ICS (Alexander et al., 2020), to support planning potential tests.

In addition to the previously discussed challenges, vulnerability ranking in ICS presents a distinct hurdle for effective penetration testing. Traditional vulnerability ranking methods, originally designed for IT environments, often overlook the potential physical consequences that may result from exploiting a vulnerability in an ICS (Stouffer et al., 2023). This gap is significant, as a vulnerability rated as low or medium severity in conventional IT terms may lead to severe operational or safety consequences when exploited in an ICS. For example, a buffer-handling flaw causing a controller reboot after receiving malformed packets may be scored as medium severity in IT terms, yet in an ICS, it could disrupt safety-critical communication and potentially trigger physical incidents. This mismatch between traditional ranking methods and the unique risk profile of ICS environments necessitates the use of more comprehensive ranking systems that incorporate physical impact and safety considerations.

Although the Common Vulnerability Scoring System (CVSS) v4.0 (FIRST, 2023) was released to address several limitations of earlier scoring approaches, its effectiveness in capturing the distinct risk profile of ICS vulnerabilities, particularly when compared to CVSS v3.1, has not yet been thoroughly evaluated through real-world ICS case studies. Furthermore, the limited availability of practical, threat modeling-based penetration testing methods for ICS, especially those leveraging insights from design-phase threat modeling, highlights the need for further research.

Based on the hypothesis that penetration testing outcomes can serve as a validation mechanism for threat modeling conducted during the system design phase, and to address the identified research gaps, this study explores the following two research questions:

RQ1: How can design-phase threat modeling guide penetration test planning for newly deployed industrial control systems?

RQ2: How effectively do vulnerability scoring methods capture the physical and safety-related consequences of vulnerabilities in industrial control systems?

Although the two research questions address different stages of the penetration testing process, they are complementary. RQ1 focuses on structuring and justifying test selection during the penetration test planning phase, while RQ2 focuses on vulnerability prioritization during the vulnerability ranking phase, which follows exploitation. Together, the two research questions support a more systematic, transparent, and traceable penetration testing workflow for ICS.

To address these research questions, this study extends the case study presented by Khalil et al. (2023) and proposes the Bridging Requirements and Implementation through Design-Guided Evaluation (BRIDGE) methodology, a nine-stage methodology for threat modeling-driven penetration testing of newly deployed ICS. The methodology builds upon the results of the threat modeling conducted during the design phase to guide penetration test planning and prioritization. The proposed methodology is validated through a real-world microgrid case study, in which both CVSS v3.1 and v4.0 were applied for vulnerability ranking.

This paper makes several key contributions to the field of ICS security:

- (1) It introduces BRIDGE, a nine-stage threat modeling-driven penetration testing methodology for newly deployed ICS. The methodology connects the design and testing phases by leveraging the outcomes of design-phase threat modeling to guide test selection and prioritization.
- (2) It extends and formalizes a mapping of MITRE ATT&CK for ICS techniques to STRIDE threat categories. This mapping bridges the gap between high-level threat analysis and intermediate-level attack representations required for penetration test planning, enabling the development of comprehensive and well-justified testing scenarios.
- (3) It evaluates the applicability of CVSS v4.0 compared to CVSS v3.1 in representing the distinctive risk profile of ICS vulnerabilities, particularly in terms of safety implications and security requirements.
- (4) It presents a real-world case study of ICS penetration testing using a newly deployed microgrid system, providing valuable insights into the practical complexities and challenges encountered in real-world projects, which are often absent in controlled laboratory environments. These insights can help practitioners better understand the realistic constraints and decision-making processes involved in ICS penetration testing.
- (5) Finally, it demonstrates how penetration testing can serve as a validation mechanism for the design-phase threat modeling outcomes, confirming that the identified requirements were correctly applied and that the system is resilient against the prioritized threats.

A preliminary version of this work was presented as a conference paper (Khalil et al., 2026). The present manuscript substantially extends that work by formalizing and naming the BRIDGE methodology, integrating the existing methodology steps into a coherent threat modeling-driven penetration testing framework, and providing an expanded case study, analysis, and discussion.

The rest of this paper is organized as follows: Section 2 discusses the background and related work; Section 3 explains the research design and introduces the BRIDGE methodology for threat modeling-driven penetration testing of ICS. Section 4 presents the results of the microgrid case study, structured according to the stages of the BRIDGE methodology. Section 5 discusses the implications of the results, while Section 6 examines the threats to the validity of the results. Finally, Section 7 concludes the study.

2. Background and related work

To ensure a secure ICS product development lifecycle, the IEC 62443-4-1 standard (Commission, 2018) defines eight key practices for secure product development. These include specifying security requirements (through threat modeling), applying secure-by-design principles (which

Table 1IEC 62443 cyber security testing stages, as described by [Commission \(2018, 2022\)](#).

Cyber Security Testing Stage	Responsible Entity	Objective	Security Verification and Validation Testing Processes
Component testing	Vendor	Verify that each component meets its operational and security requirements	Security requirements testing Threat mitigation testing Vulnerability testing Penetration testing
Integration testing	Integrator	Verify that all components work together as designed	Non-production integration tests
System testing	Might be run by third party under the system owner responsibility	To demonstrate that the system management, operational, and technical countermeasures are implemented correctly, effective, and meet the requirements.	Penetration testing Policies and procedures testing Manual security checks

involve describing how the design mitigates the identified threats), and conducting security verification and validation testing. The latter is recommended at the three stages of ICS cyber security¹ testing: component, integration, and system testing, as shown in [Table 1](#), and should be verified by the system owner. Verification testing aims to confirm that “you built it right”, for example, by verifying that the implemented system meets its defined security requirements. On the other hand, validation testing aims to ensure that “you built the right thing”, for example, by assessing whether the implemented system can withstand the prioritized threats identified during the threat modeling process ([Commission, 2022](#)).

As part of security verification and validation practices for a developed ICS component, the IEC 62443-4-1 ([Commission, 2018](#)) identifies four distinct testing processes: security requirements testing, threat mitigation testing, vulnerability testing, and penetration testing. However, it does not provide detailed guidance on how to effectively integrate and harmonize these testing efforts.

The security requirements testing process, which can be categorized as verification testing, aims to ensure that the security functions of the newly designed ICS component meet its security requirements. This testing involves various approaches, including functional testing, performance testing, and unexpected input tests to assess the product’s behavior under different scenarios. However, the threat mitigation testing process verifies that the threat mitigation techniques function as designed and validates their effectiveness in protecting the system component against the threats identified by the threat modeling. For example, variant authentication tests might be used to ensure that the implemented authentication mechanism effectively mitigates elicited spoofing threats ([Commission, 2018](#)).

The vulnerability assessment process focuses on identifying security weaknesses within an ICS component. It implies different types of testing, including abuse-case testing and known vulnerability scanning. Vulnerability assessments typically do not involve exploiting the identified vulnerabilities. However, known vulnerability scanning serves as a critical foundation for penetration testing activities. Penetration testing, in contrast, adopts a more proactive approach. It simulates real-world cyberattacks by attempting to exploit identified vulnerabilities to assess the overall security posture of a system. This enables organizations to identify and mitigate security gaps before they are exploited by malicious actors.

The IEC 62443 standard does not provide a clear description of the processes involved in the verification and validation of integration and system testing. Instead, it describes examples of processes that might be used in these two stages, as shown in [Table 1](#). It is important to

note that penetration testing was mentioned in both component and system testing. Penetration testing may serve as both verification and validation, as it verifies that the system is implemented as designed and validates that it is resilient to the threats mitigated during the threat modeling in the design phase.

ICS penetration testing overlaps with IT penetration testing in tasks similar to operating system penetration testing (e.g., Windows and Linux), as well as web application testing ([Fujimoto et al., 2023](#)). However, ICS penetration testing requires additional knowledge, as such systems use specialized protocols and standards that are not used in IT. The scope of IT penetration testing is usually limited to cyberspace, whereas attacks on ICS may result in physical impacts that should be considered during penetration testing. Also, IT system penetration testing primarily focuses on confidentiality breaches, while ICS prioritize process availability and data integrity, which should be reflected in the testing scenarios ([Fujimoto et al., 2023](#)).

Electricity is a life-sustaining service, and its disruption can significantly disrupt daily life. For this reason, infrastructure related to electricity generation and transmission was among the first critical infrastructures identified by the European Union ([Directive, 2008](#)). To the best of the authors’ knowledge, no official guideline currently exists within the European Union for the penetration testing of electric utilities. However, such a guideline has been proposed by a United States-based organization, the National Electric Sector Cyber Security Organization Resource (NESCOR) ([Searle et al., 2016](#)).

NESCOR guidelines break the penetration testing tasks of an electrical utility into five stages: *penetration test scoping*, *architecture review*, *target system setup*, *end-to-end penetration test analysis*, and *result interpretation and reporting*. The guide categorizes penetration testing tasks into four categories based on the required skills and the time required to perform each task. The *server Operating System (OS)* penetration tasks require basic testing skills, should be performed frequently, and are mostly done by internal teams. The *server application* penetration testing tasks require moderate testing skills and are commonly performed. The *network communication* penetration tasks require greater expertise, making them occasionally performed. The *embedded device* penetration tasks are advanced tasks that require highly specialized skills, and they are time-consuming ([Searle et al., 2016](#)).

While NESCOR does not explicitly refer to “threat modeling”, its recommendation to review the target system architecture through stakeholder interviews helps understand the system’s functionality, intended security posture, and the potential security risks associated with vulnerabilities. This approach aligns with the core objectives of threat modeling. In contrast, other penetration testing methodologies, such as the Penetration Testing Execution Standard (PTES) ([Penetration Testing Execution Standard, 2026](#)), explicitly incorporate threat modeling as a distinct phase. PTES defines seven stages for penetration testing: *pre-engagement interactions*, *intelligence gathering*, *threat modeling*, *vulnerability analysis*, *exploitation*, *post-exploitation*, and *reporting*.

¹ In this paper, the term cybersecurity is used in accordance with contemporary academic usage. However, when referring to official terminology from standards or organizations (e.g., IEC 62443 or the National Electric Sector Cyber Security Organization Resource), the original spelling cyber security is preserved for consistency.

Threat modeling-based penetration testing has also been proposed in several academic studies; however, most of these methodologies primarily focus on IoT systems. For example, Salzillo et al. (2020) proposes a four-stage methodology for IoT systems, consisting of *system modeling*, *threat modeling*, *planning*, and *penetration testing*. The study applies the proposed methodology to a home automation system, with a particular focus on threat modeling for the Message Queuing Telemetry Transport (MQTT) protocol,² which constitutes the paper's main contribution. It employs the Multi-Cloud Application Composition Model (MACM), a graph-based system modeling, to support threat elicitation using a threat catalog and the Common Attack Pattern Enumeration and Classification (CAPEC) (MITRE Corporation, 2023). The study provides detailed descriptions of each attack pattern of potential system threats, which are then used to formulate an effective attack plan.

Practical and agile threat research for IoT (PatrioT) is a four-stage IoT penetration testing methodology (Süren et al., 2023). The *planning*, *threat modeling*, *exploitation*, and *reporting* stages are further divided into well-documented substages. PatrioT decomposes an IoT system into seven attack surfaces: *hardware*, *firmware*, *network*, *web*, *cloud*, *mobile*, and *radio*. The methodology compiles the Open Worldwide Application Security Project (OWASP) IoT Top 10 (OWASP, 2023) into a list of 100 security weaknesses based on the Common Weakness Enumeration (CWE) (MITRE Corporation, 2026). PatrioT is designed primarily for the penetration testing of IoT products, including drones, smart fridges, and surveillance cameras.

Performing threat modeling within the penetration testing process is often a brief exercise focused on helping testers understand the system. It typically does not involve developing an extensive list of threats and often lacks information about system security requirements or the system's security posture. Design-phase threat modeling systematically addresses this by performing comprehensive threat analysis, threat prioritization, and security requirements selection for the most critical threats. However, existing penetration-testing methodologies typically treat threat modeling as an activity conducted only during the testing stage. This might be related to the typical timing of penetration testing, which is often performed long after system deployment, when secure-by-design practices may not have been applied or not documented in a way that testers can use. Consequently, the connection between early threat modeling results and penetration testing activities remains weak, leaving a methodological gap in how threats inform test selection.

Threat prioritization is a crucial stage in threat modeling conducted during the design phase. It helps justify the selection of mitigation techniques by identifying the most critical threats (Khalil et al., 2023). Similarly, categorizing vulnerabilities identified during penetration testing is essential. Vulnerability ranking facilitates prioritizing mitigation efforts and can be carried out using vulnerability assessment methods, which are typically categorized as quantitative or qualitative. Quantitative vulnerability assessment methods, such as CVSS, assign numerical scores to vulnerabilities. Whereas qualitative methods classify vulnerabilities into severity classes (e.g., high, medium, low) (Sharma et al., 2023).

CVSS is a vendor-agnostic “open framework for communicating the characteristics of software vulnerabilities” (FIRST, 2023). It has multiple versions: CVSS v2 and v3 are widely adopted, while CVSS v4.0 is the most recent release. CVSS v4.0 calculates its score based on four metric groups: *base*, *threat*, *environmental*, and *supplemental*. The *base metric* group comprises exploitability and impact metrics that remain constant over time and across user environments. The *threat metric* group may change over time, as it depends on the exploit maturity. The *environmental metric* group is related to vulnerability characteristics that depend on the user environment, similar to applied security controls. The *supplemental metric* group is defined by the user and considers additional

factors similar to the impact of the vulnerability on system safety and its resilience to recovery from the vulnerability exploit, which enhances the relevance of CVSS v4.0 for ICS environments.

Published CVSS scores are typically limited to base scores, as they are independent of customer environments. Organizations are expected to adjust these scores using the remaining metrics to produce context-specific CVSS scores. Vulnerability assessment in ICS and OT systems must consider both the potential safety impact and the system's resilience within its specific environment, factors that were not adequately addressed in earlier CVSS versions.

Several studies have applied structured threat analysis methods and adversary behavior frameworks, such as STRIDE (Howard and Lipner, 2006) and MITRE ATT&CK (Alexander et al., 2020), in ICS security research. For example, Choi et al. (2020) build an ICS testbed and use the MITRE ATT&CK framework to identify monitoring fields for generating attack datasets. They further introduce a data enrichment tool to expand the collected datasets by generating synthetic events that simulate large-scale and long-duration attack scenarios. Similarly, Tharot et al. (2023) employ ATT&CK-based game scenarios for industrial cybersecurity training, and Da Silva et al. (2023) propose an automated STRIDE-based template for the Microsoft Threat Modeling Tool in ICS contexts. While these works demonstrate the applicability of STRIDE or ATT&CK to ICS environments, they primarily focus on testbed development, training, or tooling. In contrast, this work addresses the systematic integration of design-phase threat modeling results into penetration test planning for newly deployed ICS.

Planning industrial control systems penetration testing is a challenging task due to their complexity and the wide range of technologies they often involve. Defining the testing scope and planning tests with threat modeling guidance would help identify testing priorities and support a more structured testing process. Although a limited number of studies have proposed threat modeling-based penetration testing approaches for CPS, these works typically apply threat modeling during the penetration testing stage to elicit test cases (Süren et al., 2023; Salzillo et al., 2020). However, they overlook the value of carrying forward the results of threat modeling performed during the design phase into later testing activities. Incorporating early-stage outputs, such as system models, elicited threats, and prioritization results into subsequent security activities could improve both the efficiency and effectiveness of the testing process. Moreover, there is a notable scarcity of real-world case studies that document threat modeling-based penetration testing in ICS contexts.

3. Methodology

To address the identified research gaps, this study proposes a threat modeling-driven penetration testing methodology referred to as **BRIDGE**. The methodology leverages the outcomes of the threat modeling conducted during the design phase to guide penetration test planning before the system transitions into operational use. The term threat modeling-driven is deliberately used, rather than threat modeling-based, to emphasize that the comprehensive threat modeling conducted during the system design phase plays a central role in shaping the penetration testing strategy. This approach serves as a bridge between the secure ICS design phase and its subsequent verification and validation activities.

Fig. 1 illustrates the bidirectional relationship in the threat modeling-driven penetration testing approach. Existing threat modeling-based penetration testing methods are typically unidirectional, where threat models are created during the penetration testing phase to support test planning. These methods do not incorporate or link the threat modeling efforts performed during the design stage into later testing activities. In contrast, the threat modeling-driven approach incorporates a feedback loop: penetration testing results are used to verify that the specified security requirements were correctly implemented and to validate that the threats identified during threat modeling have been effectively

² A lightweight publish-subscribe messaging protocol widely used in IoT systems.

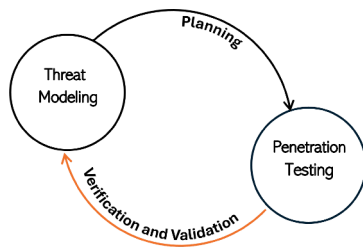


Fig. 1. The relationship between threat modeling and penetration testing in the proposed threat modeling-driven penetration testing methodology.

mitigated through the chosen requirements. This bidirectionality ensures that threat modeling is not only a planning tool but also a living artifact that is validated and refined based on testing outcomes, thereby strengthening the alignment between design-phase threat modeling and the actual security posture of the deployed system.

Fig. 2 illustrates the BRIDGE methodology for penetration testing, which consists of nine stages: (i) ICS Threat Modeling, (ii) Data Flow Diagram Update, (iii) Penetration Test Scoping, (iv) Penetration Test Planning, (v) Exploitation, (vi) Vulnerability Ranking, (vii) Reporting, (viii) Clean-up, and (ix) Report Analysis, Remediation and Retesting.

Unlike threat modeling-based penetration testing methodologies (see Section 2), the first stage of the BRIDGE methodology, **ICS Threat Modeling**, is initiated during the design phase rather than the testing phase. Conducting this stage early in the lifecycle allows the identification of potential threats before system deployment and enables the definition of critical security requirements. This stage corresponds to the comprehensive threat modeling activities performed during the system design and relies on the structured methodology for CPS threat modeling introduced in Khalil et al. (2023). The methodology provides a systematic process for identifying information and system assets, mapping the system into a Data Flow Diagram (DFD), defining the security context, determining trust boundaries, eliciting threats, analyzing their consequences and potential losses, prioritizing threats based on impact analysis, and finally deriving the system's critical security requirements. The resulting threat model forms the cornerstone of the BRIDGE methodology, supporting subsequent stages by linking high-level design insights to practical penetration testing objectives and scope definition.

The **Data Flow Diagram update** stage is initiated during the system implementation and involves discussions related to the system testing procedures. These discussions address whether component, integration, and system tests will be conducted separately, as well as the selection of the security verification and validation processes to be applied to the newly developed system component(s) (i.e., security requirement testing, security mitigation testing, vulnerability testing, and penetration testing). A decision regarding whether the tests will be executed internally or by a third party should be made as early as possible to allow sufficient time for external arrangements, if required. The DFD is updated at the conclusion of the implementation phase to ensure that no further changes to the system are introduced before testing. Following the system implementation, the cybersecurity team revises the previously identified threat list, including prioritization. It is also recommended at this stage to document both the implemented and excluded security requirements, as this facilitates test selection and supports follow-up during penetration testing result analysis.

The **Penetration Test Scoping** stage defines the boundaries and objectives of the penetration test. During this stage, stakeholders agree on the testing type (white-box, gray-box, or black-box) based on testing goals. This study advocates the use of white-box testing, particularly when the internal cybersecurity team that participated in the threat modeling phase also conducts the testing. In such cases, the process becomes more efficient and less time-consuming. Conversely, gray-box testing is often more suitable when testing is performed by external teams. Providing external testers with limited yet essential information,

such as high-level system architecture and critical information flows, enables them to concentrate on the most relevant attack vectors. While white-box testing offers deeper insight and traceability, it may present challenges for external engagements due to time constraints, confidentiality or intellectual property concerns, and higher costs associated with extended engagements.

During the scoping stage, stakeholders may agree on the complexity level of the penetration testing tasks to be included in the tests (Searle et al., 2016). This decision can be guided by the types of system components and attack surfaces, discussed in Section 2. In this study, the categorization proposed by NESCOR (Searle et al., 2016) is followed, which identifies four penetration testing task categories for electrical utilities.

The **Server OS penetration tasks** involve vulnerability assessment of Windows and Linux-based systems and aim to identify exploitable system vulnerabilities that could lead to unauthorized access. As described by Searle et al. (2016), these tasks comprise three stages: information gathering, vulnerability analysis, and exploitation. Penetration testers with limited expertise in ICS are usually familiar with these tasks, and a wide range of tools support these tests (e.g., Metasploit Rapid7, 2026).

The **Server Application penetration tasks** focus on testing applications to discover their security weaknesses. The testing activities include application mapping, application discovery, and application exploitation (Searle et al., 2016). These tests are generally more complex and time-consuming than the server operating system tasks.

The **Network Communication penetration tasks** aim to exploit vulnerabilities in network protocols and communication mediums. They comprise two subcategory tasks: radio frequency packet analysis and network protocol analysis. These tasks are complex, as they require in-depth knowledge of network protocols and may involve specialized tools, such as those used for wireless attacks. This is particularly relevant, as ICS environments, including electric utilities, rely on diverse wireless technologies, such as Wi-Fi, cellular networks, and the Global Positioning System (GPS). Protocols such as Zigbee (ZigBee Alliance, 2015), Modbus (Modbus Organization, 2012), and DNP3 (DNP Users Group, 2005) are also widely used in ICS networks, and attacks against them require a solid understanding of protocol datagrams and functionality.

Embedded Device penetration tasks include electronic component analysis, field technician interface analysis, and firmware binary analysis, and are the most demanding and time-consuming penetration testing tasks. They involve testing embedded and microprocessor-based devices, including firmware, input interfaces such as serial and parallel ports, as well as non-volatile data storage such as flash memory and EEPROM, which typically store firmware and configuration data (Searle et al., 2016).

Deciding which task categories to include in the testing scope sets expectations regarding the testing timeframe, required expertise, budget, and expected outcomes. Excluding one of the mentioned four task categories implies accepting the possibility of undiscovered vulnerabilities within that category and requires explicit approval from the relevant stakeholders.

Another key decision during the scoping stage involves selecting which threats to include in the testing plan. Threats identified during the threat modeling exercise and updated during the **DFD update** stage often form a long list of threats that vary in severity and consequences. Selecting a prioritized subset of threats for testing is therefore essential for a focused and effective penetration test. This prioritization should be based on explicit criteria, such as criticality or expected impact type, so that testers can concentrate on the most consequential threats and optimize the testing effort and its outcomes.

The scoping stage should also include discussions of the targeted domains and IP ranges, the specific systems and environments to be tested, and the level of access to be granted. It should define the types of attacks to be included or excluded from testing, such as DoS and social engineering. Since the tests are conducted before the system is

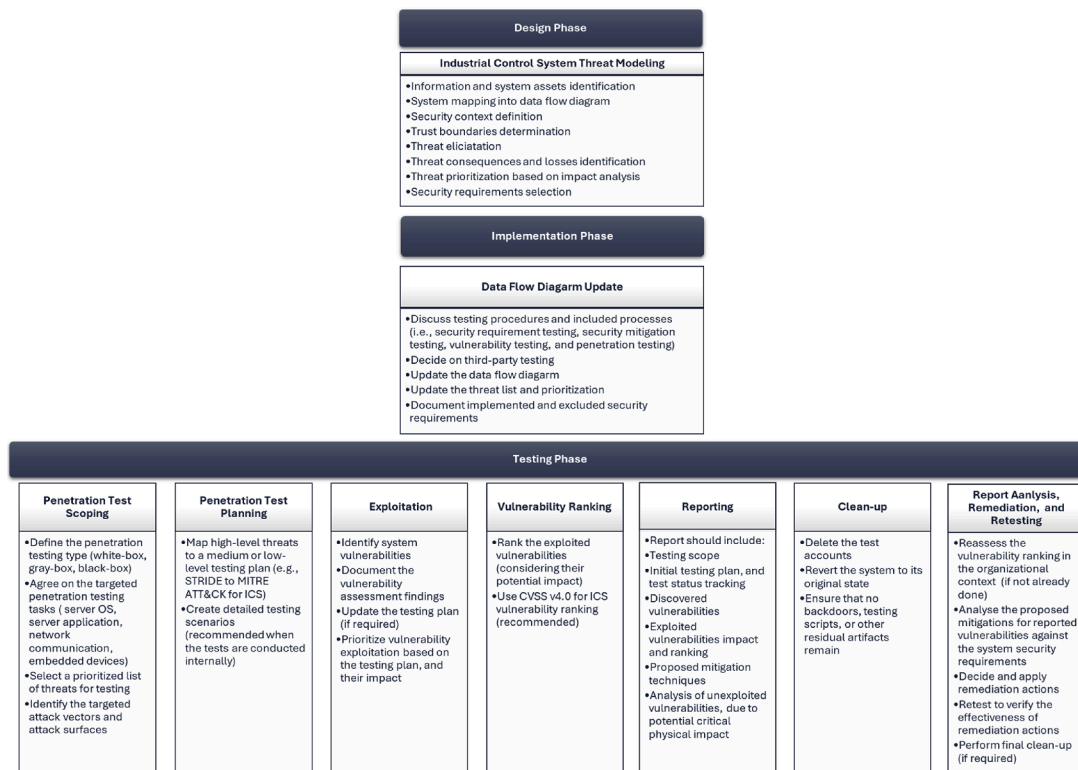


Fig. 2. BRIDGE: A threat modeling-driven penetration testing methodology for newly deployed industrial control systems.

transitioned to operations, social engineering attacks might not be considered, as no operators are yet involved. For DoS testing, the decision to include or exclude such tests should depend on their potential consequences, as some tests may cause service interruptions or component failures even in a non-operational environment. In addition, the scope should clearly identify the attack surfaces relevant to the ICS environment, such as remote access interfaces, internet-facing applications, and potential lateral movement paths within the network. These discussions help establish realistic testing boundaries and minimize the likelihood of unintended disruptions.

The **Penetration Test Planning** stage is carried out based on the previously defined testing scope. During this stage, testers may rely on supporting frameworks and reference guides to structure the testing plan. Guides, such as the NESCOR testing guide (Searle et al., 2016) and STRIDE threat trees (Howard and Lipner, 2006), provide high-level guidance for the test planning without detailed testing procedures. More detailed procedures are available in domain-specific resources such as the OWASP testing guides (OWASP, 2026), which cover web security, mobile application, and firmware security testing methodology.

In practice, penetration test planning approaches vary across organizations and projects and are strongly influenced by operational, safety, and availability constraints, particularly in environments involving operational technology (Stouffer et al., 2023). Existing guidance emphasizes that all penetration testing engagements should begin with careful planning and scoping to ensure that testing activities do not adversely impact operational systems (Searle et al., 2016). A key contribution of this study is to demonstrate how an ICS penetration testing plan can be guided by STRIDE threats mapped to MITRE ATT&CK for ICS techniques (see Section 4.3.2), supporting structured and risk-informed test selection. Depending on the agreed-upon timeframe and available resources, testers may prepare a detailed plan that defines testing scenarios and procedures. Alternatively, when testing is performed by third-party teams, as in the case study presented in Section 4, the planning

stage may result in a high-level plan that outlines the techniques to be tested while leaving the detailed technical procedures to the tester.

The **Exploitation** stage is guided by the testing plan. Testers identify system vulnerabilities and execute the planned tests while prioritizing the exploitation based on the plan and potential impact. They document their findings throughout the process, updating the testing plan as needed to address newly discovered attack vectors. During the **Vulnerability Ranking** stage, exploited vulnerabilities are analyzed and prioritized based on their potential impact. This may involve using CVSS or a custom ranking system tailored to the ICS environment. Based on the case study results, presented in Section 4, this study recommends using CVSS v4.0 for ICS vulnerability ranking due to its consideration of the impact of the vulnerability on the system safety. The vulnerability ranking stage ensures that the mitigation efforts focus on the most critical vulnerabilities.

Testers should document all findings in a comprehensive report during the **Reporting** stage. The report may follow the OWASP guide (Saad and Mitchell, 2020) by mentioning the test scope, limitations, timeline, and an executive summary of the test objectives, key findings, and recommendations. The findings section should also follow the best practices (Saad and Mitchell, 2020), and include: Vulnerability type and rating, impact, root cause (e.g., code bug, configuration issue, security flaw), used testing technique(s), and remediation. The successful tests should be documented in detail, allowing the reproduction of the test results. In the context of threat modeling-driven penetration testing, this study recommends that the report include a briefing about the initial test plan (which may be added as an appendix) and highlight which tests were successful, failed, or excluded due to time limitations or unfulfilled pre-conditions.

For systems that require an IEC 62443 security level four,³ It is further recommended to report all discovered vulnerabilities,

³ “Prevent the unauthorized disclosure of information to an entity actively searching for it using sophisticated means with extended resources, Industrial

including those that were not exploited during the testing period. The report should also include an analysis of critical unexploited vulnerabilities, as in ICS environments, some vulnerabilities may pose a significant risk of physical damage to equipment or have human safety implications. In such a case, testers should prioritize safety by avoiding full exploitation of the vulnerability. Instead, a proof-of-concept demonstration should be conducted to validate the vulnerability and provide sufficient technical details to help stakeholders understand its potential impact, consequences, and possible mitigation measures.

When penetration testing is outsourced to a third party, it may be difficult to obtain the level of detail recommended by this methodology in the final report. However, clearly defining the expected content and level of detail during the initial negotiations with the testers can help ensure that the final report includes the required level of information. The three stages (**Exploitation**, **Vulnerability ranking**, and **Reporting**) can be executed in parallel or sequentially, depending on the project's complexity and available resources.

Following the reporting stage, it is essential to initiate a **Clean-up** stage to ensure the test environment is fully restored. All accounts created for testing purposes should be deleted, system configurations should be reverted to their original state, and any files or artifacts left on compromised assets should be removed. Placing the clean-up stage after the reporting stage allows stakeholders time to verify and review the report findings before the system is reset. This sequencing is consistent with best practices recommended in the professional penetration testing literature,⁴ alongside the subsequent remediation and retesting stage.

The BRIDGE methodology concludes the penetration testing process with the **Report Analysis, Remediation, and Retesting** stage, where the reported vulnerabilities are re-assessed within the context of the organization, particularly if contextualization was not fully addressed during the reporting stage. If penetration testing is prioritized over security requirements testing and threat mitigation testing within the verification and validation processes, then the cybersecurity team should analyze the proposed mitigations for reported vulnerabilities, identified through penetration testing, against the system security requirements. Stakeholders may also review the testing plan status to check whether tests were successful, failed, initially excluded, or excluded during the exploitation stage due to time or technical limitations. This review helps the stakeholders identify whether any threats from the prioritized list were not tested during the exploitation stage.

Remediation actions for the reported vulnerabilities should then be identified and implemented, followed by verification and validation testing to ensure the applied measures are effective. Retesting is especially critical for high and critical severity vulnerabilities, as they may lead to a significant impact if left unresolved before the system becomes operational. If necessary, a final clean-up round should be conducted after retesting to remove any temporary remediation or diagnostic elements introduced during the testing process.

The BRIDGE threat modeling-driven penetration testing methodology provides a structured approach for ICS penetration testing, bridging the design and testing phases. By addressing the unique ICS challenges through threat modeling, vulnerability prioritization, and comprehensive reporting, the methodology enhances the effectiveness and efficiency of security assessments, ultimately contributing to a stronger security posture for ICS.

4. Microgrid case study

This study demonstrates the application of the BRIDGE threat modeling-driven penetration testing methodology through a real-world

case study of a newly developed microgrid system. Since the threat modeling for this microgrid, conducted during the design phase, was discussed in detail in Khalil et al. (2023), this paper focuses on the results of the proposed penetration testing methodology, starting from the second stage: Data Flow Diagram Update.

Sections 4.1–4.3 collectively address RQ1 by presenting the results of applying the BRIDGE methodology to the microgrid case study. Section 4.5 addresses RQ2 by comparing the severity ranking of the reported vulnerabilities using CVSS v3.1 and CVSS v4.0. Finally, Section 4.8 evaluates the research hypothesis defined in Section 1 by validating it against the results of the case study.

The penetration testing process involved a collaborative effort among five stakeholders: the cybersecurity team (including the first and second authors); the power systems expert (the third author), representing the power engineering team and the system owner; the software architect; and the third-party penetration tester. The penetration testing process was conducted over eight months, beginning in February 2023 and concluding in September 2023. During this period, stakeholders participated in 13 technical meetings to discuss the details of each stage. These meetings did not include managerial discussions related to the selection of the testing entity or contractual agreements.

4.1. Data flow diagram update

As a reminder of the microgrid system introduced in Khalil et al. (2023), the hardware structure of the analyzed system includes a microgrid control unit, mapped as the Server in the DFD. This unit consists of computational equipment and a software platform and is responsible for data and log management, as well as other services such as monitoring. The Server is based on an in-house software platform designed and implemented by the project's research teams.

The system also includes an Energy Storage System (ESS), which is used for storing excess energy and supplying it when required. The ESS can be used to maximize the utilization of locally produced energy and to provide capacity management, as well as ancillary services such as power quality improvement, voltage support, and system restoration. The Moulded Case Circuit Breaker (MCCB) is a power system component required to connect the microgrid to the Distribution System Operator (DSO) grid through the Point of Common Coupling (PCC). The system further includes PhotoVoltaic (PV) generation and may facilitate one or more prosumers, which are proactive entities that either produce or consume energy.

In the DFD presented in Khalil et al. (2023), energy consumption was measured using energy meters, while operational measurements of power system properties were performed using multimeters. Meter data concentrators and PV data concentrators, implemented as in-house software components, were used to collect and aggregate data from different parts of the system and communicate it to the Server.

During the DFD updating stage, the cybersecurity team collaborated with the software architect and the power systems expert to discuss design changes implemented since the initial DFD creation. These discussions revealed several system design modifications, which were reflected in the updated version of the DFD, shown in Fig. 3. Compared to the DFD created during the design phase and discussed in Khalil et al. (2023), all energy meters were replaced with multimeters. Unlike an electric energy meter, a multimeter is capable of measuring multiple parameters and enables both energy metering and the measurement of electrical properties such as voltage and current. Consequently, the “meter data concentrator”, which was intended to host an in-house software application for collecting data from energy meters, was removed.

The updated DFD also reflects a refinement of the trust-zone definitions introduced during the design-phase threat modeling. As a result of these refinements, Trust Zone 3 was eliminated, and the prosumer multimeter was relocated from Trust Zone 1 to Trust Zone 2. In addition, a new Trust Zone 4, representing the OT network, was added as a result

Automation and Control System (IACS) specific skills and high motivation” (Commission, 2019).

⁴ <https://www.coresecurity.com/blog/six-stages-penetration-testing>

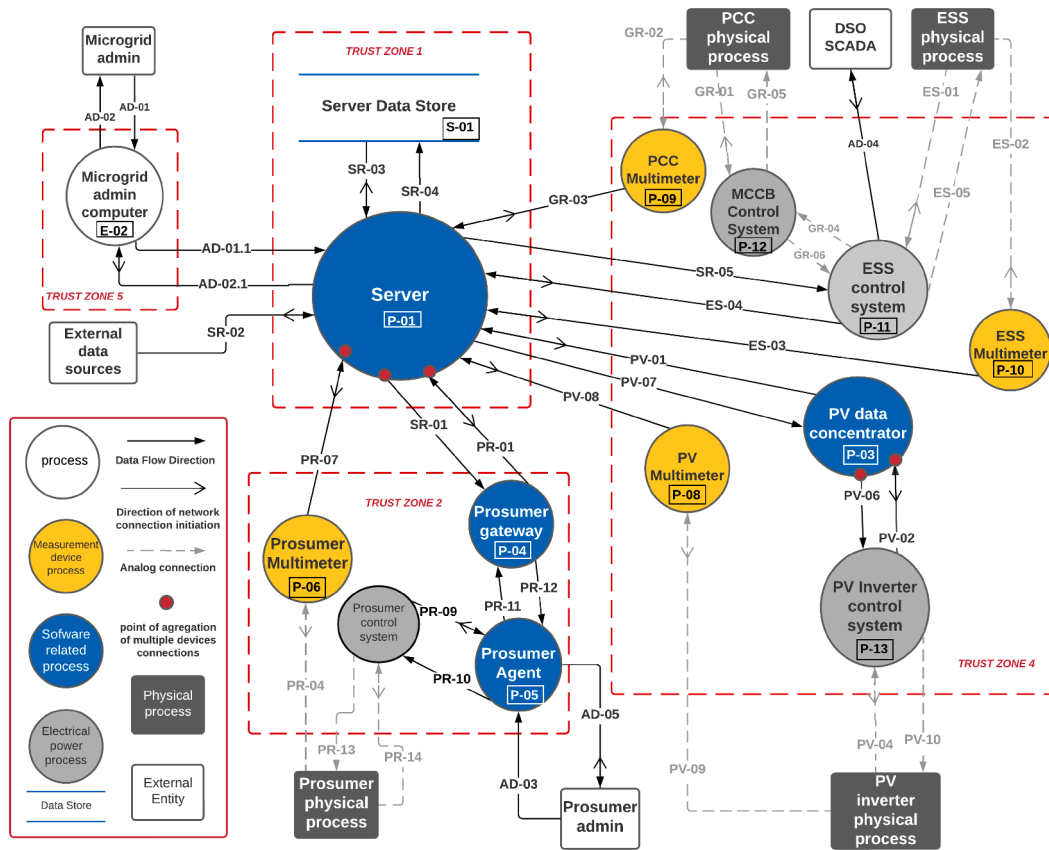


Fig. 3. The updated microgrid data flow diagram.

of the implemented segmentation. Furthermore, in the initial DFD, the microgrid administrator computer was modeled as an external process. However, this representation was corrected in the updated DFD, as the microgrid administrator workstation is managed by the same organization operating the microgrid and does not constitute an external entity in the strict sense. Instead, the administrator computer was placed in a separate trust zone to reflect a different security posture and access pattern, while still being treated as an internal process under organizational control.

The DFD updates significantly impacted the list of 110 threats identified during the design phase. The changes resulted in the suppression of 19 threats deemed irrelevant and the introduction of three new threats. Subsequently, the elicited threat list was refined, and the prioritization was re-evaluated based on these updates.

Considering resource and time constraints, the stakeholders prioritized penetration testing within the IEC 62443 security verification and validation testing processes. As a result, security requirements and threat mitigation tests were excluded from the project. Negotiations to determine whether the penetration testing would be conducted internally by the cybersecurity team or outsourced to a third party were concluded during the system implementation phase.

The stakeholders decided to involve a third-party penetration tester, ensuring a fresh perspective on the system's security posture. Engaging a third party with specialized penetration testing experience and skills aimed to support the effective management of the limited testing time. This approach ensured better resource management, given the limited number of internal cybersecurity experts involved in this project.

Negotiations with the third-party penetration tester proceeded in parallel with the DFD updates. The final agreement was reached with a professional freelance penetration tester with extensive experience in IT system penetration testing and a working background in OT systems. Once the agreements and DFD updates were finalized, the tester was provided with the updated DFD, data flows, and the prioritized threat

list. Sharing the threat modeling outputs at this stage was intended to support the tester's understanding of the system's threat landscape and potential impacts. Follow-up meetings were then scheduled to review the provided documents in detail and address any questions raised by the tester.

To address the penetration tester's specific inquiries regarding the network protocols used within the microgrid system, the data flows mapping table, originally described in Khalil et al. (2023), was updated to include the protocol associated with each data flow, as illustrated in Table 2. Unlike traditional software data flow diagrams, which typically represent data exchange between software processes, an ICS DFD mostly illustrates information flows between physical devices and sub-systems. Therefore, annotating the network protocols used for each data flow provided valuable context for the penetration tester, potentially facilitating the identification of vulnerabilities specific to those protocols. The DFD update stage was intended to ensure that all relevant documentation was updated and that the tester fully grasped the data flow diagram and critical system threats.

4.2. Penetration test scoping

Following the documentation updates and the engagement of the penetration tester, stakeholder meetings were held to define the penetration testing scope. The penetration testing type was agreed upon as a gray-box test, where the DFD, high-level network diagrams, and threat lists were provided to the tester, along with addressing the tester's inquiries. However, the source codes were not disclosed.

The stakeholders then discussed the complexity of the penetration testing tasks and agreed to prioritize testing of OS, server applications, and network communication. The scoping decisions adopted for the penetration testing activities were driven by the consequence and loss assessment performed during the design-phase threat modeling, as presented in Khalil et al. (2023). In particular, priority was given to the

Table 2

Example of updated entries in the data flow mapping table, with updates highlighted in bold.

Flow ID	From	ID	To	ID	Information Asset Type	Network Protocols
PV-01	PV data concentrator	P-03	Server	P-01	Operational Data*	Modbus TCP
PV-02	PV inverter control system	P-13	PV data concentrator	P-03	Operational Data*	Modbus RTU over RS485

*Operational Data: data representing the operational status of assets.

in-house developed software components, especially the Server, which functions as the microgrid control unit and hosts internet-facing services. The impact assessment conducted during the design phase (Khalil et al., 2023) showed that spoofing attacks targeting the Server could lead to severe system-level consequences, including exceeding operational limits and misleading safety operations. These consequences may escalate into safety-critical losses, including loss of life or injury to people.

Embedded device testing was deprioritized during this testing phase due to time and resource constraints and because the embedded components were supplied by established ICS vendors that apply baseline security controls. This scoping decision does not reduce the criticality of the findings, as compromise of the in-house software could result in severe safety and availability impacts on the microgrid.

The scoping stage discussions further included a review of the list of critical threats and DFD elements to be included within the testing scope, together with their associated consequences. Potential attack scenarios for the selected DFD elements were brainstormed, considering how such attacks would manifest in the system and how the penetration tester could verify their success. The importance of critical threats guiding the testing plan was emphasized while allowing the tester flexibility to explore beyond the predefined threat list.

The meetings then focused on potential attack surfaces (e.g., internet-facing web applications), attack vectors (e.g., unpatched vulnerabilities, weak credentials), threat actors (e.g., malicious actors), and possible attack scenarios. The system architecture places the most safety-critical OT components within a segmented OT network (Trust Zone 4), where they are protected from direct internet access. Accordingly, the penetration testing effort prioritized externally reachable attack surfaces that, based on the impact assessment, could enable an internet-based adversary to influence critical system functions. The tester was first expected to attempt access to the system through publicly accessible interfaces and applications. When this was not feasible within the agreed testing timeframe, controlled access to Trust Zone 1 was provided to allow the assessment to proceed. The OT network itself remained within scope, and the tester was explicitly challenged to attempt to reach it through network-based attacks without being granted direct access.

Discussions also addressed the testing location, ultimately deciding on remote testing while excluding attacks requiring physical presence on-site. Social engineering attacks, such as phishing emails, were also excluded as they rely on user interaction, which was outside the scope of this initial testing phase since the system was not yet operational.

Stakeholders then discussed how the tester would gain initial access to each zone, considering the specific attack vectors relevant to each zone and the security measures in place to limit access. A list of required tester access (e.g., Virtual Private Networks (VPN), jump host) and permission levels (e.g., regular user, admin) was created. To allow the tester to observe the impact of attacks on system alerts, regular user and administrator access were provided to specific tools.

The discussions also addressed the depth of testing, considering whether to focus solely on initial device access or include attempts of lateral movement within zones. Ultimately, it was agreed to provide the tester with limited-access accounts that allowed initial access to selected devices. This approach enables the tester to assess the possibility of lateral movement within the system. Additional discussions included testing time frames, critical vulnerability reporting procedures, and the protocol for handling DoS attacks.

In summary, the penetration test scoping stage defined the testing type, boundaries (i.e., out-of-scope activities), critical attacks to be executed and their consequences, rules of engagement, emergency contacts, and expectations about the penetration testing timeframe.

4.3. Penetration test planning

The cybersecurity team was keen on structuring the planning of the penetration testing based on an attack database that complements the penetration tester's knowledge. However, the selection of a suitable attack database was a major challenge. During the design phase, the microgrid attack taxonomy proposed in Bahsi et al. (2022) was updated. This high-level taxonomy proved valuable for threat elicitation, as described in Khalil et al. (2023). However, it lacked specific details on attack execution, which is crucial for planning penetration testing. For this reason, the use of STRIDE threat trees (Howard and Lipner, 2006) was investigated as an attack database for this study. The threat trees were discussed in detail during two meetings with the penetration tester. While these trees provided a structured approach, their primary focus on software threats created significant gaps in coverage of ICS-specific attacks. This limitation would have resulted in an incomplete penetration testing plan.

STRIDE is considered a high-level threat analysis model (Strom et al., 2018), suitable for threat elicitation during the design phase, where detailed system specifications (e.g., protocols, vendors, OS) are unavailable. However, such information becomes available after system implementation and during penetration test planning. Recognizing the limitations of both the high-level taxonomy, used in Khalil et al. (2023), and STRIDE threat trees, alternative approaches were explored to achieve a more detailed level of modeling that includes richer attack execution details.

As the tests were performed by a third party, the cybersecurity team aimed to guide the testing plan without imposing specific tools and scenarios, as this is typically the responsibility of the tester. For this reason, it was decided to use MITRE ATT&CK for ICS, a mid-level modeling framework that provides details about attack techniques (Strom et al., 2018). However, MITRE ATT&CK for ICS is not designed specifically for penetration testing and lacks detailed information on how to execute the tests. While it identifies attack techniques, the penetration tester would still need to identify the specific tools and exploit each technique.

Additionally, since the threat elicitation was done using STRIDE, mapping these threats to the tactics and techniques of MITRE ATT&CK for ICS became crucial. At this stage, it was decided to have focused meetings between the cybersecurity team and the penetration tester, where MITRE ATT&CK for ICS was discussed in detail and mapped to STRIDE threats. These discussions aimed to guide the penetration tester in planning the tests in a structured way. The following subsections describe the process of selecting potential tests based on MITRE ATT&CK for ICS and illustrate the procedure for mapping STRIDE categories to MITRE ATT&CK for ICS techniques.

4.3.1. Attack preconditions

As discussed in Khalil et al. (2023), to cause harm to a system, except for certain types of DoS attacks, an attacker must first gain access to the system, which is considered an attack precondition. Accordingly, three attack precondition stages were defined: *Initial Access*, *Discovery*, and *Lateral Movement*, as shown in Fig. 4. These stages are also defined as tactics in the MITRE ATT&CK for ICS. In a possible attack scenario,

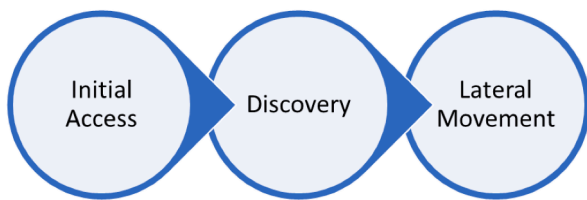
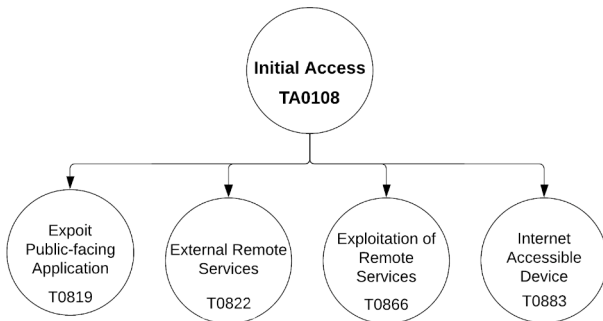


Fig. 4. Attack precondition stages.

Fig. 5. MITRE ATT&CK for ICS *Initial Access* techniques targeted in the case study testing plan.

the tester gains initial access to the system using one of the *Initial Access* techniques (e.g., *exploit of public-facing application*) to attack the system from the internet. Then, the tester can proceed with system discovery using the *Discovery* techniques (for example, using *network sniffing*) and perform *Lateral Movement* (using *default credentials* or sniffed *valid accounts*).

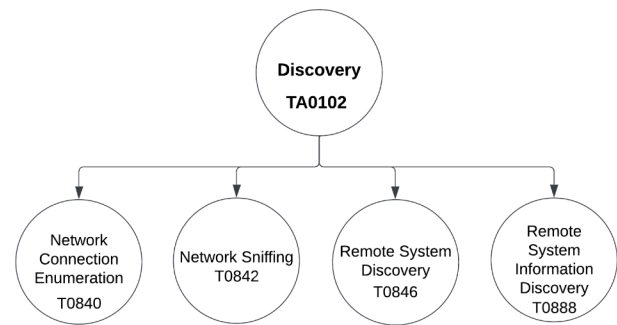
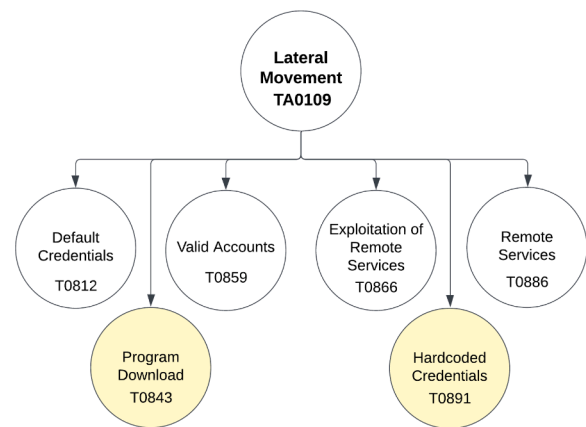
In addition to the STRIDE categorization of MITRE ATT&CK for ICS techniques (see Section 4.3.2), possible precondition techniques that the tester might use to gain access to the system were also selected. Figs. 5, 6, 7 present only the selected precondition techniques for this case study. The full list of techniques related to each of the three chosen tactics can be found in the MITRE ATT&CK matrix for ICS.⁵

To gain initial access, it was agreed that the penetration tester would simulate an attacker attempting to compromise the system from the internet. Accordingly, four techniques were selected from the MITRE ATT&CK for ICS *Initial Access* tactic, as shown in Fig. 5. The *exploit public-facing application* and *exploitation of remote services* leverage software vulnerabilities to gain unauthorized access to the system. The *external remote services* technique targets services similar to VPN, potentially using valid accounts. While the *internet accessible device* technique aims to gain access to internet-exposed devices, intentionally or unintentionally, such as cellular modems, without the use of exploits.

The *remote services* technique was excluded as it targets internal services typically used by operators for remote management, making it more relevant to lateral movement within the context of this study. Additionally, MITRE ATT&CK for ICS *Initial Access* techniques requiring the tester's physical presence (e.g. *rogue master*), user interaction (e.g., *spearphishing attachment*, *drive-by compromise*), or those outside the case study scope (e.g., *supply chain compromise*) were also excluded from testing.

It was agreed that if remote access to the system is successfully gained from the Internet, the tester may proceed with network discovery to identify further vulnerabilities that could be exploited for lateral movement. However, if all attempts using the selected *Initial Access* techniques are unsuccessful, the tester may use the provided limited-access accounts to establish initial access and initiate the network discovery.

Four techniques were selected from the *Discovery* tactic, as shown in Fig. 6. The *network connection enumeration* might be used by the pen-

Fig. 6. MITRE ATT&CK for ICS *Discovery* techniques targeted in the case study testing plan.Fig. 7. MITRE ATT&CK for ICS *Lateral Movement* techniques targeted in the case study testing plan.

etration tester to inspect the state of network connections and collect information about devices' roles. The *network sniffing* is a fundamental technique to collect information about the system. The technique might also be used to collect information about the used subnets and internal domains. The *remote system discovery* and the *remote system information discovery* techniques might be used to discover the system's remote services and collect information about these services, such as the configuration.

Following the *Discovery* stage, the tester might attempt to pivot between devices on the same network using *Lateral Movement* techniques. It was previously agreed that the testing scope encompasses lateral movement simulations across distinct network segments. The testing plan focused on four core techniques selected from the *Lateral Movement* tactic (as shown in Fig. 7), with an additional two techniques identified as potential candidates for further testing (highlighted in yellow).

Information gathered during discovery about devices and operating systems can be used to identify the manufacturer's default passwords, enabling the tester to employ the *default credentials* technique for lateral movement. Similarly, enumerated user or service accounts during discovery can be leveraged for the *valid accounts* technique. The *exploitation of remote services* technique, previously used for initial access, remains applicable for lateral movement within the system. Additionally, the *remote services* technique might be used to navigate the network laterally.

The *program download* technique was marked as a potential test due to its specific applicability to ICS environments. This technique often requires initial access to the OT network (Trust Zone 4) as a precondition. Similarly, the *hardcoded credentials* technique was identified for potential testing, depending on the available time frame. Exploiting these credentials might involve leveraging vulnerabilities to reveal them, which may require advanced system knowledge.

⁵ <https://attack.mitre.org/matrices/ics/>

4.3.2. STRIDE-based categorization of MITRE ATT&CK for ICS techniques

Following the definition of attack preconditions, the next step focused on mapping the MITRE ATT&CK for ICS techniques to the STRIDE threat categories, to facilitate alignment between the threats identified during the threat modeling phase and the penetration testing plan. This approach helped to prioritize and assess the relevance of each categorized technique within the testing plan. However, this process also revealed that not all MITRE ATT&CK for ICS techniques represent threat types that can be meaningfully expressed using STRIDE. For example, several *Execution* techniques, such as those involving command-line and graphical user interfaces, describe how an unauthorized actor (or, in this case, the penetration tester) interacts with the system. These techniques could serve as attributes within one of the created STRIDE categorizations, but they do not fit neatly into the core STRIDE categories. Therefore, such techniques were not included in the categorization, which focused solely on those that could be clearly associated with one or more STRIDE threats.

Similarly, the *Command-and-Control* tactic in MITRE ATT&CK for ICS describes adversary communication mechanisms and operational behaviors rather than security threats in themselves. As such, these techniques do not map directly to STRIDE threat categories and are instead treated as enabling techniques that support the realization of STRIDE threats. In the same way, the *Impact* tactic in MITRE ATT&CK for ICS describes the consequences of successful attacks rather than the threats or adversary actions themselves. Therefore, all techniques included in the *Command-and-Control* and *Impact* tactics were excluded from this categorization as well.

The procedure used for categorization is illustrated in Fig. 8. The first author mapped the MITRE ATT&CK for ICS techniques using MITRE ATT&CK Navigator⁶ by reviewing each technique description and annotating the corresponding STRIDE category (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, or Elevation of privilege) within the Navigator.

Some techniques were classified under multiple STRIDE categories. For example, the *adversary-in-the-middle* technique allows an attacker to intercept traffic between two nodes and collect sensitive information about the exchanged data. In such cases, the attacker could also modify packets, inject false data, or block traffic from reaching its intended destination. For this reason, the *adversary-in-the-middle* technique was categorized under information disclosure, tampering, spoofing, and denial of service. The *rogue master* technique was also categorized under spoofing and tampering, as it refers to a malicious controller that impersonates a legitimate controller to issue control commands and may also tamper with them.

The first author then initiated a meeting between the cybersecurity team and the penetration tester to discuss the proposed categorization and agree on the techniques to be included in the test plan. For each technique, a decision was made regarding its inclusion or exclusion, and the feasibility of executing each selected technique was further examined.

It should be noted that the initial categorization was performed using MITRE ATT&CK for ICS v13, while the information was later updated to align with v18, which was the latest version available at the time of writing. MITRE ATT&CK for ICS v18 includes 83 techniques, of which 46 were directly mapped to STRIDE threat categories. Techniques that were applicable to more than one STRIDE threat type were counted only once in this total. When additionally considering techniques relevant to the defined attack preconditions (discussed in Section 4.3.1), including all techniques from the *Initial Access*, *Discovery*, and *Lateral Movement* tactics, and excluding duplicates already captured within the STRIDE categorization, the total number of potential techniques considered for ICS penetration test planning amounted to 64 out of the 83 techniques.

Some techniques require significantly more time and effort than others and may also depend on certain preconditions that must be met. For example, executing an attack based on the *unauthorized command message* technique would first require gaining initial access to the OT network (Trust Zone 4). Without this access, the penetration tester would be unable to perform the corresponding test. Such advanced techniques were marked for potential inclusion, depending on the availability of time and fulfillment of necessary preconditions. The remainder of this section details the proposed STRIDE categorization of MITRE ATT&CK for ICS techniques.

Spoofing: Eight techniques were categorized as spoofing, as shown in Fig. 9. The spoofing tests included four core techniques (*default credentials*, *adversary-in-the-middle*, *valid accounts*, *execution through API*) to be executed during the exploitation stage. Three techniques were identified as potential tests, to be performed if time and access permitted: *masquerading* and *hardcoded credentials* (conditional on time), and *unauthorized command message* (conditional on gaining initial access to the OT network, Trust Zone 4, which was not provided to the tester). The *rogue master* technique was excluded from the testing scope, as the assessment focused on attacks that can be launched remotely, without physical access to the system. Finally, to prioritize testing of the in-house control software, techniques relevant to that software, similar to *valid accounts* and *execution through API*, were given precedence.

It is to be noted that the *execution through API*, one of the techniques under the *Execution* tactic, was mapped to spoofing only in scenarios where API calls could be issued without proper authentication or where an adversary could impersonate a trusted system component, thereby violating identity guarantees.

During the threat modeling phase, the most critical threats were spoofing threats. Using the proposed spoofing categorization for MITRE ATT&CK for ICS techniques, the tester may simulate an attacker's behaviors. For instance, the tester may combine the *adversary-in-the-middle* technique with the use of *valid accounts* to mimic a threat agent who gains unauthorized access and impersonates a Microgrid administrator, potentially disrupting critical operations. Additionally, the tester can evaluate the security of API endpoints by attempting to execute commands and assessing whether authentication is required. Techniques such as using devices' default passwords (i.e., *default credentials*) or brute-forcing accounts (i.e., *valid accounts*) can also be employed to identify potentially weak authentication mechanisms.

Tampering: Eighteen techniques were categorized as tampering, as detailed in Fig. 10. However, only the *adversary-in-the-middle* was explicitly included in the tampering testing plan. The categorization of tampering techniques sparked interesting discussions. Techniques similar to *block reporting message* might seem more suited for DoS, since it disrupts the monitoring services. However, it was also included in the tampering category. Consider a scenario where a system configuration always displays the last received monitoring data without raising alarms for missing reports. In such a case, blocking reporting messages during an attack could be considered tampering with the system's ability to report accurate information. The same logic applies to *block command message*. For example, if an attacker blocks a message containing a new setpoint value (a desired operating condition), the system will not receive the update and will remain on the previous setting. This effectively tampers with the system configuration established by the administrator.

Other tampering techniques were de-prioritized as it was considered that the successful execution of the *adversary-in-the-middle* technique could serve as a proof of concept for techniques such as *block reporting message* and *block command message*, if the testing timeframe did not allow those tests to be performed. The *system firmware* and *module firmware* were excluded as the embedded device attack surface, including firmware, lay outside the scope of this plan. The *Rootkit*, *rogue master*, *exploitation for evasion* techniques were also excluded.

It should be noted that the *modify controller tasking* technique, part of the *Execution* tactic, was introduced in a later update of the categorization and was therefore incorporated into Fig. 10. This technique

⁶ <https://mitre-attack.github.io/attack-navigator/>

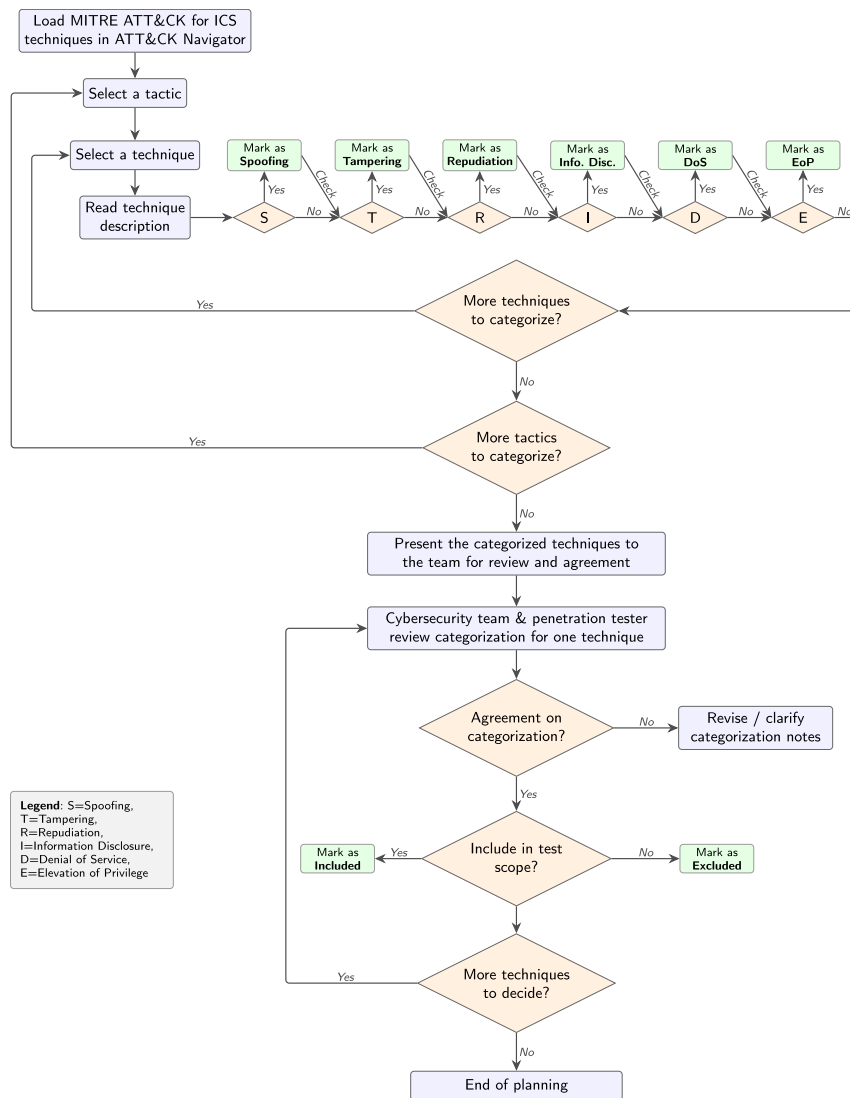


Fig. 8. Description of the process used to categorize MITRE ATT&CK for ICS techniques into STRIDE threat categories.

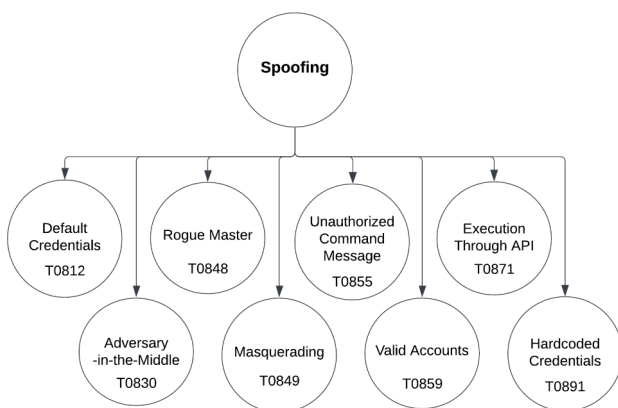


Fig. 9. Proposed categorization of Spoofing techniques for MITRE ATT&CK for ICS.

represents tampering due to the unauthorized modification of program task associations and execution properties, which directly alter controller behavior. As the penetration testing plan was defined prior to

this update, *modify controller tasking* was not considered during the testing plan.

Reputation: Six techniques were categorized as repudiation techniques, as shown in Fig. 11. However, it was decided to exclude all the techniques related to repudiation from the testing plan as the losses and consequences of the repudiation threats were minimal compared to other threats. The repudiation techniques mainly aim to hide the attacker's traces, which was set out of the scope of the testing plan, as the main goal was to simulate attacks that might cause a critical impact on the system.

Information Disclosure: Eleven techniques were categorized as information disclosure techniques, as shown in Fig. 12. Notably, the eleven techniques categorized under Information Disclosure all fall within the *Collection* tactic of MITRE ATT&CK for ICS, indicating a consistent alignment between the two classifications. During the threat modeling, information disclosure threats were classified as low-priority due to their limited impact on system functionality. However, four information disclosure techniques were still included in the testing plan: *adversary-in-the-middle*, *data from information repositories*, *data from local system*, and *screen capture*. While in real attack scenarios, the *screen capture* technique is mostly combined with malware infection or other automation techniques, it is a vital tool in a penetration testing

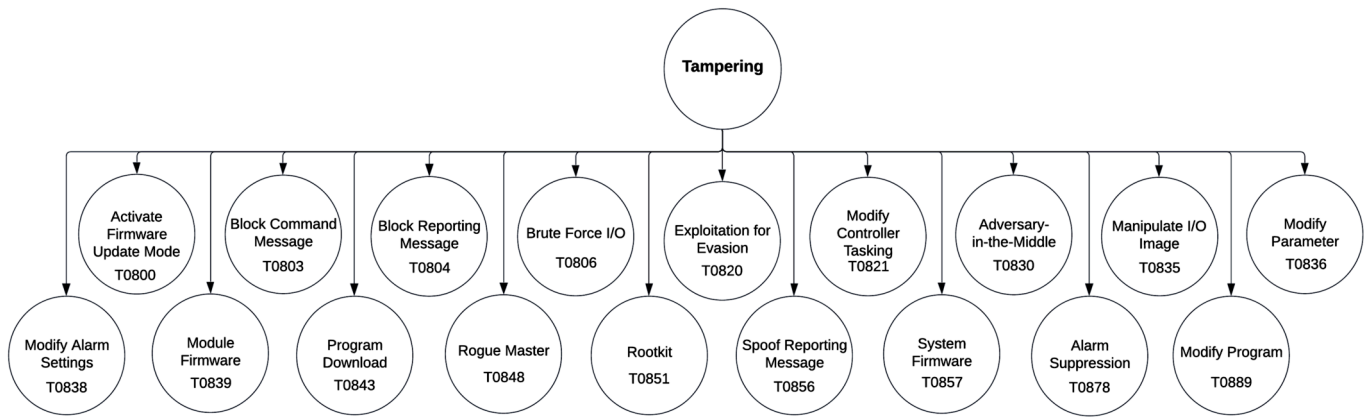


Fig. 10. Proposed categorization of Tampering techniques for MITRE ATT&CK for ICS.

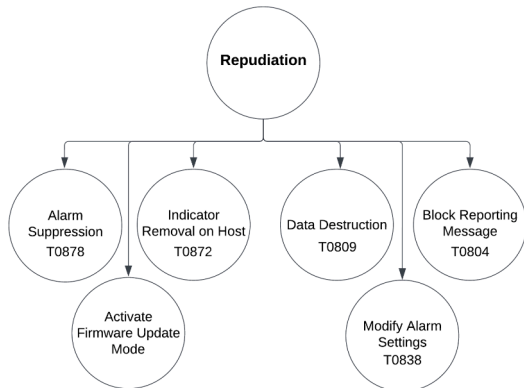


Fig. 11. Proposed categorization of Repudiation techniques for MITRE ATT&CK for ICS.

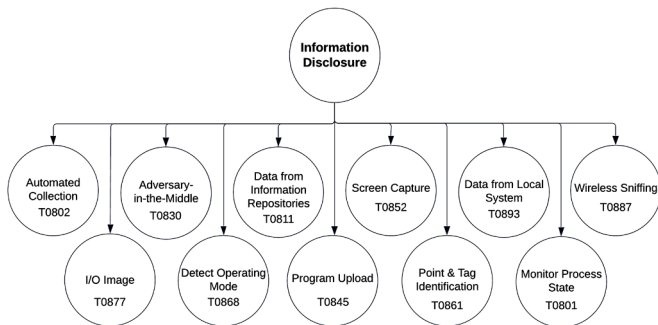


Fig. 12. Proposed categorization of Information Disclosure techniques for MITRE ATT&CK for ICS.

exercise as well. It documents the tester's accomplishments and the types of information exposed during the simulated breach. The primary goal in incorporating these four techniques was to understand the potential information an attacker could access and how the collected information may help in an attack progression.

Five information disclosure techniques, however, were deprioritized: *I/O image*, *detect operation mode*, *program upload*, *point & tag identification*, and *monitor process state*. These techniques require gaining access to the OT network (Trust Zone 4), a prerequisite not granted for this testing phase. The *automated collection* technique was excluded due to the potential time commitment required for scripting and automation tool development, and given the low priority of the information disclosure threats. *Wireless sniffing* was also excluded, as there is no wireless network in the target system.

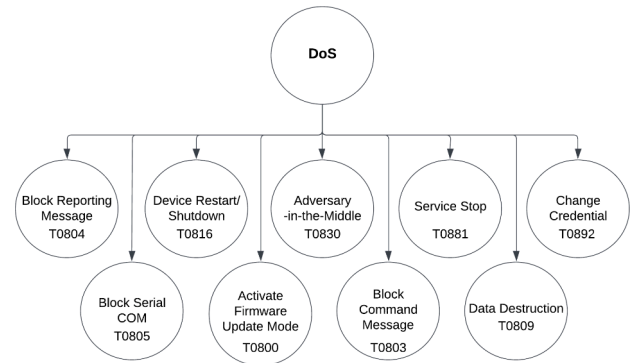


Fig. 13. Proposed categorization of Denial of Service techniques for MITRE ATT&CK for ICS.

Denial of Service: Nine techniques were categorized as DoS, as illustrated in Fig. 13. Since the target system was not yet operational during testing, the *adversary-in-the-middle* technique was selected for execution to evaluate the system's resilience to denial-of-service conditions using a controlled and non-destructive testing approach. Successfully performing this technique was considered sufficient to demonstrate the feasibility of other related DoS techniques that rely on intercepting or blocking communication. For example, if the tester successfully spoofs the default gateway's MAC address using a device that cannot process such traffic, this should result in a denial of service. The use of the *adversary-in-the-middle* technique in such a case proves that other techniques similar to *block reporting message* and *block command message* are also possible. The *block serial com* technique was excluded from testing. The remaining techniques were retained as potential testing candidates if sufficient time was available and the necessary preconditions were met during the assessment, provided that they could be executed using controlled and non-destructive testing approaches.

Elevation of Privilege: Finally, eight techniques were categorized as elevation of privilege techniques, as illustrated in Fig. 14, reflecting the updated categorization aligned with MITRE ATT&CK for ICS v18. The *exploitation for privilege escalation* and *valid accounts* techniques were included in the penetration testing plan, with the potential to add the *hooking* and *change of operation mode* techniques if the tester gained access to the OT network. The *system firmware* technique was excluded, as discussed in the tampering categorization.

It should be noted that three techniques shown in Fig. 14, *autorun image*, *system binary proxy execution*, and *native API*, were incorporated during a later update of the categorization when aligning the study with MITRE ATT&CK for ICS v18. The first two techniques were introduced in later ATT&CK versions, while *native API* already existed in v13 but was not previously categorized under Elevation of Privilege. As these

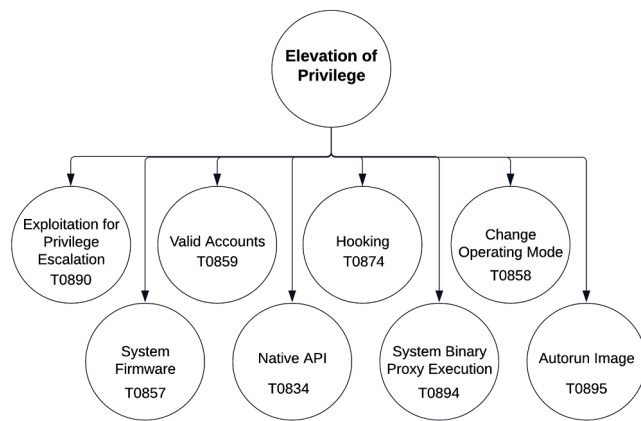


Fig. 14. Proposed categorization of Elevation of Privilege techniques for MITRE ATT&CK for ICS.

techniques were not part of the original penetration test planning, they were not included in the executed testing scope. They are mapped to Elevation of Privilege only when they lead to unauthorized privilege escalation through the abuse of trusted system mechanisms.

The planning stage aimed to create a high-level test plan that considered the prioritized system threats. When the tests are performed by a third party, such a plan should guide the testers through the exploitation stage, while giving them the freedom to choose the testing tools and procedures. However, for tests performed internally, the plan could be further extended to include more detailed testing procedures, with commands to be used and tools.

4.4. Exploitation

The main goal of this case study is to discuss the penetration testing process, with a primary focus on the planning. The third party fully conducted the exploitation stage without the involvement of the cybersecurity team. Furthermore, the tests were performed on a real-world, operational system. For these reasons and considering the confidentiality of the exploitation stage results, they are excluded from the scope of this paper.

4.5. Vulnerability ranking

Twelve software vulnerabilities were reported by the penetration tester. Notably, not all of the reported vulnerabilities were directly related to traditional OT devices, although they could still have severe impacts on the system's safety and availability. As previously described, the primary testing focus was the in-house software platform responsible for microgrid management, which, although implemented as software, operates as a critical OT component within the system architecture. Consequently, several findings reflect software and web-exposed weaknesses that could still affect operational safety and continuity. These included, for example, weak credentials, SQL injection, unencrypted communications, and authentication-related misconfigurations identified on management and communication interfaces.

The tester categorized the reported vulnerabilities using the CVSS v3.1 calculator⁷ based on the base metric. The severities of these vulnerabilities were rated as three critical, two high, three medium, and four low. As CVSS v4.0 incorporates safety and resiliency aspects, the first author later re-categorized the same vulnerabilities using the CVSS v4.0 calculator,⁸ while considering the supplemental, environmental, and

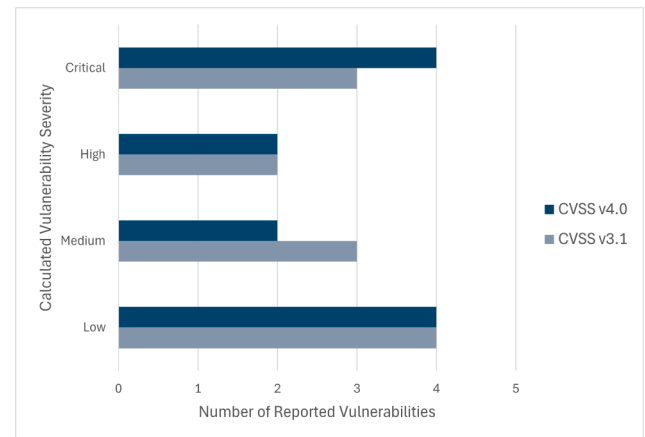


Fig. 15. Comparison of the severity ratings assigned to reported vulnerabilities using CVSS v3.1 and CVSS v4.0.

threat metrics.⁹ This resulted in a revised severity distribution of four critical, two high, two medium, and four low vulnerabilities, as shown in Fig. 15. Overall, the observed score shifts were primarily driven by the incorporation of subsequent system impact, system security requirements, and safety-related consequences in CVSS v4.0. Severity differences between the two versions were minimal for vulnerabilities with limited operational implications.

Table 3 provides a detailed comparison of selected vulnerabilities to illustrate how contextual factors in CVSS v4.0 influence severity assessment. Vulnerability number one represents a grouped finding in which multiple technical weaknesses were reported collectively, resulting in high confidentiality, integrity, and availability impact values under CVSS v3.1. Accordingly, it received a base score of 8.8 (High). When re-evaluated using the CVSS v4.0 base metrics, the score remained comparable at 8.7 (High), since no impact on subsequent systems was identified.

However, when applying the CVSS v4.0 environmental metrics, the system context identified during the requirement selection phase was explicitly incorporated. The Confidentiality Requirement (CR) had been determined as Low during the threat-modeling exercise, while the Integrity Requirement (IR) and Availability Requirement (AR) were set to High to reflect operational priorities. As this grouped vulnerability predominantly affected confidentiality aspects and did not introduce safety-relevant consequences, the adjusted CVSS v4.0 score slightly decreased to 8.6 (High). This example illustrates how CVSS v4.0 enables contextual refinement of severity based on system security requirements rather than relying solely on generic technical impact assumptions.

A key distinction in CVSS v4.0 is the explicit modeling of subsequent system impact and safety-related consequences. Vulnerability number two in Table 3 corresponds to a demonstrated denial-of-service attack against a system component. During the threat-modeling phase, denial of service on this component had already been associated with operational consequences such as interruption of control functions, limited energy availability, insufficient power availability, and potential loss of energy sold. Under CVSS v3.1, the vulnerability received a base score of 7.4 (High) primarily due to the availability impact on the affected component. When evaluated using CVSS v4.0 base metrics, Availability impact on both the Vulnerable System (VA) and the Subsequent System (SA) was set to High to reflect the modeled downstream operational effects, resulting in an increased base score of 8.3 (High). Upon incorporation of environmental and safety-related metrics, including recognition

⁷ <https://www.first.org/cvss/calculator/3.1>

⁸ <https://www.first.org/cvss/calculator/4.0>

⁹ This action was performed during the report Analysis, remediation, and retesting stage (see Section 4.8), although the results are presented under the vulnerability ranking section to facilitate readability and comparison.

Table 3

Severity comparison between the CVSS v3.1 base score and the CVSS v4.0 base and adjusted scores for a sample of reported vulnerabilities.

CVSS v3.1 Base Score Calculator										CVSS v4.0 Calculator																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
Vulnerability number	Base Score								CVSS v3.1 Base Score	Base Metrics											CVSS v4.0 Base Score	Supplemental Metrics										Environmental (Modified Base Metrics)						CVSS v4.0 Score																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
										Exploitability Metrics				Vulnerable System Impact Metrics			Subsequent System Impact Metrics										Exploitability Metrics			Vulnerable System Impact Metrics			Subsequent System Impact Metrics			Environmental (Security Requirements)			Threat Metrics																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																								
1	Adjacent (A)	Low (L)	None (N)	None (N)	Unchanged (U)	High (H)	High (H)	High (H)	8.8 High	Adjacent (A)	Low (L)	None (N)	None (N)	None (N)	None (N)	High (H)	High (H)	High (H)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)	None (N)

of potential safety implications arising from loss of control or energy imbalance, the final CVSS v4.0 score increased to 9.2 (Critical). This case demonstrates how CVSS v4.0 enables severity escalation grounded in previously identified system-level consequences rather than solely in localized technical impact.

Vulnerability number three in Table 3 further illustrates the effect of safety-related contextualization in CVSS v4.0. This finding involved a missing authentication weakness on a component within the OT network (Trust Zone 4). Although initially rated 5.4 (Medium) under CVSS v3.1, the potential safety and operational implications associated with unauthorized access in this zone resulted in an adjusted CVSS v4.0 score of 7.9 (High). The increase reflects the model's ability to capture safety-relevant consequences in operational environments, even when the original technical impact appears moderate.

The analyzed examples demonstrate that CVSS v4.0 provides structured mechanisms to both refine and escalate vulnerability severity in alignment with system requirements and threat consequences. Findings with limited safety and operational impact may experience contextual reduction, while vulnerabilities associated with downstream operational disruption and safety-relevant effects may justifiably escalate in criticality. This case study demonstrates the practical effectiveness of CVSS v4.0 in accurately ranking ICS vulnerabilities, particularly by highlighting safety-related consequences. Based on these results, the study strongly recommends adopting CVSS v4.0 for vulnerability assessments within industrial control systems.

4.6. Reporting

The penetration tester provided a report documenting the testing methodology, the scope of the tests, and the assumptions made. The report also included details on the discovered vulnerabilities (including proof of concept and CVSS v3.1 base scores) and corresponding mitigation recommendations. The structure of the submitted report followed a conventional vulnerability-class-based format aligned with widely adopted testing guides. Following the report submission, two meetings were held with the penetration tester to review the identified vulnerabilities and discuss their impacts. Based on these discussions, the tester adjusted the CVSS v3.1 scores and resubmitted a revised version of the report.

As the submitted report did not include detailed information about the failed and unperformed tests, a follow-up meeting was conducted

with the penetration tester to document the testing results comprehensively. During this meeting, each technique was evaluated to determine whether it had been executed successfully or unsuccessfully, as well as those excluded due to time constraints or unmet preconditions. To enhance clarity, a color-coded scheme was applied to the categorized techniques to distinguish between the different outcomes. This visual documentation covered all techniques identified during the planning stage (See Section 4.3).

Fig. 16 presents an illustrative example of this color-coded approach. While selected penetration testing results are discussed in other sections, this figure is provided solely to demonstrate how the proposed documentation approach can be applied, as the complete results cannot be disclosed due to confidentiality constraints associated with the operational system.

In this spoofing techniques example, the *rogue master* technique was initially excluded during the planning stage. The *default credentials* and *adversary-in-the-middle* techniques were successfully executed during the testing phase, while the attempt to use the *valid accounts* technique was unsuccessful. The figure also shows that the *unauthorized command message* and *execution through API* techniques were excluded as the prerequisite conditions, such as access to the OT network, were not met. Additionally, the *masquerading* and *hardcoded credentials* techniques were omitted due to time limitations. This documentation approach enhances the transparency of the testing process and can assist in planning future exercises on the same platform.

4.7. Clean-up

The clean-up stage involves reverting the system to its original state by the tester, ensuring that no backdoors, testing scripts, or other residual artifacts remain. In most cases, the penetration tester's involvement concludes with the reporting stage, at which point all accounts created for testing purposes must be deleted.

4.8. Report analysis, remediation, and retesting

The clean-up and the report analysis, remediation, and retest stages were added as separate steps in the methodology based on lessons learned from this case study, given their sensitivity and the potentially high impact if they are not properly executed. The reassessment of vulnerability rankings using CVSS v4.0 was performed after the final report

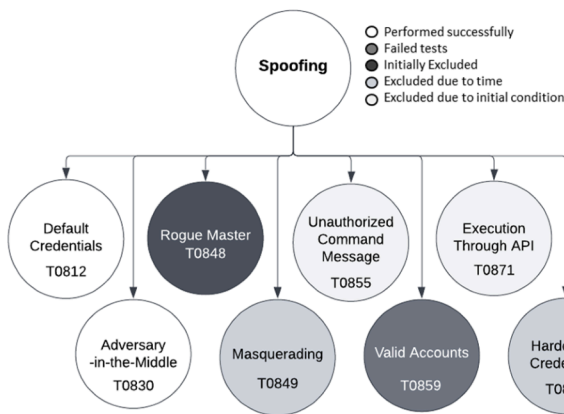


Fig. 16. An illustrative example of the color-coded approach using MITRE ATT&CK for ICS categorized spoofing techniques.

was submitted; however, its results are presented in the vulnerability ranking section (Section 4.5) to facilitate readability and enable direct comparison with the CVSS v3.1 scores. As part of this reassessment, the cybersecurity team retested selected vulnerabilities and communicated with stakeholders to confirm that all testing accounts had been deleted and that the proposed remediation had been implemented on the system.

As the penetration testing report identified six critical and high vulnerabilities, a subsequent analysis was conducted to validate the research hypothesis by examining the reported vulnerabilities and their required mitigation strategies. This analysis compared the proposed mitigation strategies for each reported vulnerability against the security requirements established during the design phase. The goal was to determine whether the proposed mitigation strategies were already addressed by the security requirements and to identify any potential requirement gaps. Additionally, the analysis assessed whether the security requirements had been properly considered during the design phase, or whether they were overlooked or not implemented correctly.

As an illustrative example of how the reported vulnerabilities were analyzed and traced, one of the identified findings involved an SQL injection flaw. Such attacks may be categorized as either *Information Disclosure* or *Tampering*, depending on their resulting impact on the system. Within the MITRE ATT&CK Enterprise matrix, SQL injection is explicitly mapped to *Exploit Public-Facing Application (T1190)*. In contrast, the technique with the same name in the MITRE ATT&CK for ICS matrix, *Exploit Public-Facing Application (T0819)*, does not explicitly reference SQL injection. This distinction reflects the higher level of abstraction adopted in the ICS matrix, which emphasizes operational effects rather than specific IT exploitation mechanisms. It also highlights the necessity of extending the categorization process to incorporate relevant techniques from the Enterprise (and, where applicable, Mobile) matrices to ensure more comprehensive technique coverage, given that ICS environments include both IT and OT systems.

In the reported vulnerability, the tester successfully exploited the SQL injection vulnerability to extract valid user account credentials from the compromised system component. This action constitutes unauthorized access to stored credential data and may align with the *Data from Local System (T0893)* technique in the ICS matrix, which was categorized under information disclosure (See Fig. 12). The extracted credentials were later used to authenticate as a legitimate user, which aligns with the *Valid Accounts (T0859)* technique. The tester then performed a spoofing attack on the compromised component. This attack demonstrates a threat scenario that was discussed during the planning phase.

The potential threat consequences associated with spoofing an administrative account of this component were already identified during the threat elicitation conducted during the design phase (Khalil et al., 2023). By successfully masquerading as the component administrator,

an attacker would obtain authorized access to privileged control functions within the system. This access could enable the modification of operational parameters (e.g., setpoints, protection thresholds, load balancing configurations), alteration of grid-connection or disconnection commands, manipulation of energy dispatch logic, disabling of alarms, or interference with monitoring and visibility functions.

the relevant threat consequences that were identified during the design phase, included overload or exceeding operational limits, interruption of energy transfer, Interrupting/manipulating the control, inability to provide ancillary services, misleading safety operations, limited energy availability, misleading connection/disconnection operations, violation of power quality, insufficient power availability, loss of energy sold, insufficient visibility of inverters, and leaking information for further attacks. Although the vulnerability originates at the software level, the resulting attack chain demonstrates how compromise of administrative credentials may propagate across control logic and operational processes, ultimately leading to severe operational and safety consequences in an ICS environment.

During the threat-modeling phase, the selected security requirements included IEC 62443-4-2 component requirement (CR) 3.5, *Input Validation*. CR 3.5 requires that components validate the syntax, length, and content of data received via external interfaces. Proper implementation of this requirement is expected to mitigate injection-based attacks, including SQL injection. Therefore, the existence of the reported vulnerability indicates insufficient enforcement or improper implementation of input validation controls.

Additional requirements, such as CR 1.1, *Human User Identification and Authentication*, including Requirement Enhancement (RE) 1, *Unique identification and authentication*, and CR 1.7, *Strength of Password-Based Authentication*, including RE (1), *Password generation and lifetime restrictions for human users*, were also specified during the design phase. However, the exploitation revealed that the extracted credentials corresponded to a weak password configuration, indicating inadequate enforcement of password policy controls.

While the primary entry point of the attack was the input validation weakness, the presence of weak credential management significantly increased the exploitability and impact of the vulnerability. Accordingly, the root cause of the attack reflects both insufficient input validation and ineffective enforcement of authentication hardening measures, demonstrating a breakdown in defense-in-depth implementation rather than a deficiency in requirement selection.

Another reported category was vulnerable software. Unlike authentication flaws, this finding does not correspond to a specific STRIDE threat category; rather, it represents an attack precondition that may enable subsequent exploitation techniques, such as exploitation of remote services or public-facing applications within the MITRE ATT&CK for ICS matrix. During the requirement selection phase, IEC 62443-4-2 CR 3.10 *Support for Updates* was included to ensure that system components provide the technical capability to perform upgrades and updates. However, IEC 62443-4-2 does not define requirements for structured patch management processes or mandatory update cycles; such governance-level controls are addressed separately in IEC TR 62443-2-3 (IEC, 2015), which provides guidance on patch management processes in industrial automation and control system environments. Therefore, the presence of vulnerable software reflects deficiencies in lifecycle and update governance practices rather than an omission in the selected component-level security requirements.

The qualitative review revealed that mitigation techniques for nine of the twelve reported vulnerabilities had already been defined through the selected security requirements. In these cases, proper implementation and enforcement of the identified controls would have prevented or significantly limited the respective attacks. The vulnerable software finding reflected lifecycle and patch governance deficiencies beyond the scope of the selected component-level requirements. The remaining two low-severity vulnerabilities were not explicitly prioritized during the threat-modeling phase due to their limited operational impact;

however, their identification still required corrective action by the system owner, reflecting one of the core objectives of performing system-level penetration testing. These results align with the research hypothesis that penetration testing can serve as a validation mechanism for design-phase threat modeling. The findings further underscore the importance of conducting security requirements testing and threat mitigation testing before initiating penetration testing. Such preparatory work increases the likelihood of identifying higher-impact vulnerabilities during testing, particularly when time constraints limit the depth of third-party assessments.

Comprehensive reporting and subsequent analysis of the penetration testing results provided valuable insights into the system's security posture. The report, together with the review meetings, ensured a clear understanding of the performed tests, discovered vulnerabilities, and their corresponding mitigation strategies. Ultimately, this process validated the relevance of several security requirements while also identifying critical gaps in their implementation. These results further demonstrate the effectiveness of threat modeling-driven penetration testing as a structured approach for the security verification and validation testing of an ICS.

5. Discussion

Threat modeling is an analytical design-phase process intended to identify critical threats and propose mitigation strategies. However, the threat modeling process by itself does not guarantee that the resulting system will achieve a minimum acceptable level of security. Even when security requirements are derived during threat modeling and communicated to the development team, practical constraints such as budget limitations, time pressure, or implementation challenges may lead to incomplete or incorrect implementation of these requirements.

The IEC 62443 standard outlines four distinct testing processes for security verification and validation testing of a newly designed system component: security requirements testing, threat mitigation testing, vulnerability testing, and penetration testing, where each process is executed independently. In such a setting, security requirements testing and threat mitigation testing should precede penetration testing, along with a more extensive vulnerability assessment beyond known vulnerabilities. Penetration testing would then focus on assessing the residual attack surface of the implemented system rather than validating individual security requirements.

However, in small and medium-sized ICS projects with limited resources, such as the project examined in this case study, where IEC 62443 certification was not targeted, stakeholders may decide to combine these processes to streamline the security assessment. Due to resource constraints, this case study prioritized penetration testing over security requirements testing and threat mitigation testing. Nevertheless, the security engineering activities were conducted with the intent of aligning with IEC 62443 principles, despite the constraints of a real-world project.

Several penetration testing methodologies integrate a threat modeling phase to support penetration test planning, hereafter referred to as threat modeling-based methodologies. Methodologies such as PatIoT (Süren et al., 2023) and PTES (Penetration Testing Execution Standard, 2026) extend into the exploitation stage by providing structured guidance on how to derive attack paths and execute them. Such approaches focus on exploitation, as their primary objective is to guide the tester in carrying out technical attack procedures.

In contrast, the objective of the threat modeling-driven penetration testing methodology proposed in this study is not to prescribe detailed exploitation steps but to bridge the system design phase with penetration testing activities. The BRIDGE methodology focuses on guiding penetration test planning based on the identified critical threats, and may also be used, when IEC certification is not aimed, to verify that security requirements have been correctly implemented and to validate that threats identified for mitigation have been effectively addressed. In this

Table 4

Comparison of BRIDGE, a threat modeling-driven penetration testing methodology, with existing threat modeling-based penetration testing methodologies.

Aspect	PTES	PatIoT	BRIDGE
Integration of design-phase threat modeling	✗	✗	✓
Threat modeling supports penetration test planning	✓	✓	✓
IoT-oriented	✗	✓	✗
ICS-oriented	✗	✗	✓
Security requirements verification and validation	✗	✗	✓
Provides technical procedures	✓	✓	•

Symbols: ✓ = present; • = partial or medium-level coverage; ✗ = not provided.

sense, the proposed approach is also verification- and validation-driven. Importantly, methodologies that provide exploitation procedures can still complement the BRIDGE methodology in the exploitation stage, since BRIDGE does not prescribe how exploitation should be technically performed, but instead ensures that penetration testing is systematically aligned with the threats identified during threat modeling and the corresponding security requirements defined during system design. The key distinctions between threat modeling-based methodologies and the BRIDGE methodology are summarized in Table 4.

Documenting security requirements derived from prioritized threats in alignment with IEC 62443 is part of the first phase of the BRIDGE methodology and was addressed in previous work on the same micro-grid project (Khalil et al., 2023, Section 5.9). As the present study focuses on penetration test planning, the full set of security requirements was not required for the planning and execution of the testing activities and was therefore not shared with the penetration tester. Instead, as described in Section 4.1, the tester was provided with the updated system model, the data flow diagram, and prioritized threats together with their anticipated consequences. This reflects common industrial practice, where the validation of individual security requirements remains outside the scope of penetration testing. Following test execution, the penetration testing findings were retrospectively analyzed by the internal cybersecurity team against the design-phase security requirements (Section 4.8) to assess whether these requirements had been effectively addressed.

Even in a fully compliant certification process, penetration testers would not require direct access to the security requirements, as these are addressed and verified through earlier, independent testing activities. While this separation of testing processes could not be fully realized in the present resource-constrained project, the documentation produced across the BRIDGE methodology preserves traceability and would support a more extensive, certification-driven testing process if pursued in the future.

The results of this case study highlight a potential need for prioritizing security requirements alongside threat prioritization during the design phase. While prioritizing threats is crucial, it can result in an extensive list of security requirements, especially when multiple requirements are assigned to each threat. This may obscure which requirements are truly critical. To address this, prioritizing security requirements themselves can help ensure that, when multiple security requirements are provided to a single threat, the most critical requirements receive appropriate attention throughout the design process. Additionally, negotiating and following up on the implementation of security requirements with system owners helps ensure that those associated with critical and high-priority threats are properly implemented.

Accordingly, the results of this study suggest that security requirements testing and threat mitigation testing should be performed before conducting penetration testing whenever possible. Running these two processes before penetration testing may reveal critical vulnerabilities that were supposed to be mitigated by design, allowing system owners to address these vulnerabilities before the penetration testing. This, in turn, would enable penetration testers to focus on more advanced

attack scenarios. In such cases, the testers may discover critical attack vectors that were not considered or were low-priority during the threat modeling phase. These vulnerabilities may result from unforeseen system interactions, configuration weaknesses, or zero-day vulnerabilities. Hence, the penetration testing results can further validate or refine the threat priorities identified during threat modeling. If a seemingly low-priority threat is shown to have a critical impact during testing, it can be addressed with greater urgency.

In practice, engaging a third party for penetration testing offers valuable expertise, but it also has limitations. While the cybersecurity team may strive to guide the testing plan to encompass both IT and OT security testing, a key challenge lies in achieving the right balance between IT and OT testing activities. Maintaining high-level involvement during the planning stage is recommended for effective collaboration with external testers. However, outsourcing the testing to a third party limits the ability of the cybersecurity team to define a detailed testing plan. High-level planning with the tester can ensure that the critical threats are tested. However, the tester may still prioritize certain types of tests over others. Overlooking OT-specific tests may potentially result in a bias towards IT vulnerabilities compared to those specific to the OT environment. Nevertheless, this case study demonstrates that IT vulnerabilities can still have a significant impact on ICS safety.

Consistent with these practical constraints, the vulnerabilities revealed in this microgrid case study were primarily software-related in nature. The identified vulnerabilities correspond to common software-level vulnerability classes typically assessed in IT penetration testing, such as authentication, cryptographic, and application-level weaknesses, even when these vulnerabilities are present on OT components. The predominance of such vulnerabilities can be attributed to the focus on remote, network-based testing and the practical constraints of limited testing time. Nevertheless, the analysis showed that these software-level vulnerabilities could still have critical impacts on the OT system, as they may expose control-relevant components and lead to safety and availability consequences.

The case study highlights the advantages of CVSS v4.0 for ICS vulnerability scoring. Compared to CVSS v3.1, CVSS v4.0 offers a more effective assessment by incorporating system security requirements (confidentiality, integrity, and availability) and the vulnerability's impact on the safety of subsequent systems. These factors lead to a more relevant and accurate CVSS v4.0 score for ICS environments.

Due to the inherent complexities of a real-world case study, the research encountered challenges that redirected the investigation in unforeseen directions. While many CPS threat modeling methods exist, STRIDE was selected for the threat modeling phase as it offers a balance between simplicity and clarity. STRIDE focuses on basic cybersecurity concepts, which makes it easy to assimilate for stakeholders with limited or no cybersecurity background. Notably, during the early design phase, a high-level threat modeling method was required to support the choice of security requirements. A more detailed taxonomy could have elicited a broader range of threats, which could also increase the complexity and time commitment of the threat modeling exercise, potentially hindering its use in future projects. In contrast, penetration test planning requires a middle-level taxonomy that provides sufficient detail about ICS attack types while remaining manageable for high-level plan creation, especially since the testing was conducted by a third party. These practical constraints justify the need to map MITRE ATT&CK for ICS techniques to STRIDE threats.

This paper focuses on linking design-phase threat modeling outcomes with subsequent security testing activities to improve traceability across the secure development life cycle. Mapping STRIDE threat categories to MITRE ATT&CK for ICS techniques enables design-phase threats to be systematically carried forward into penetration test planning. Both STRIDE and MITRE ATT&CK are qualitative, expert-driven frameworks; therefore, the proposed mapping is intentionally interpretive rather than rule-based. Certain MITRE ATT&CK for ICS techniques are associated with multiple STRIDE categories when they contribute to

different threat objectives, reflecting real-world attack behavior rather than enforcing a strict one-to-one mapping. By explicitly documenting these overlaps, exclusions, and cases where the mapping is based on explicit justification, the approach improves transparency and supports reproducibility.

The case study further revealed that threat modeling-driven penetration testing requires a reporting structure that differs from traditional vulnerability-class-based formats. Unlike conventional reports, such as those structured around OWASP testing classes, the BRIDGE methodology requires findings to be explicitly mapped to elicited threat scenarios, associated system components, and the relevant techniques used during the exploitation phase, while documenting the attack results (as shown in Fig. 16). Moreover, as ICS environments may involve web, IoT, and mobile interfaces, testing techniques may span multiple frameworks beyond a single guide. Accordingly, future applications of BRIDGE should formally define a reporting template that supports cross-framework technique integration and preserves traceability between design-phase threat modeling outputs, penetration test planning decisions, and exploitation results.

The inherent confidentiality of ICS penetration testing reports, due to the sensitive nature of system design and vulnerabilities, poses a challenge for academic publication. In this case study, the penetration testing procedures are discussed without disclosing specific system details, such as network protocols, vendors, or device models. Information about the system components targeted by the tests, successful and failed tests, and detailed descriptions of the identified vulnerabilities are also omitted. Instead, the STRIDE-based categorization of MITRE ATT&CK for ICS techniques is presented, highlighting how this categorization can guide the tester. Additionally, general information about some vulnerability categories (e.g., software, DoS) is provided to support reader comprehension while preserving confidentiality. This approach protects the operational system from potential targeted attacks based on the level of information provided in this paper.

Taken together, the findings of this case study highlight both methodological and practical contributions for threat modeling-driven penetration testing in ICS environments. By balancing confidentiality with knowledge sharing, the BRIDGE methodology provides a valuable reference for future threat modeling-driven penetration testing projects of newly deployed ICS. In addition, there is also a notable scarcity of resources that comprehensively describe the real-world execution of ICS threat modeling-based penetration testing, including what types of information should be discussed at each stage and which decisions should be made. This paper aims to contribute to this knowledge base by documenting these aspects alongside the case study results. As a result, the findings not only validate the methodology but also offer practical guidance for practitioners facing similar constraints in real ICS deployments.

6. Threats to validity

This section outlines the limitations of the proposed methodology and the case study, and identifies potential threats to the validity and generalization of the reported results.

The proposed methodology is intended for planning the initial penetration test of a newly deployed ICS rather than serving as a general-purpose penetration testing framework. Certain methodological choices made during the case study, such as the mapping of MITRE ATT&CK techniques to STRIDE categories, may be applicable beyond the evaluated system. However, it is important to keep in mind that the categorization was developed specifically in the context of penetration testing and was informed by expert discussions within the project team.

Some of the technique categorizations may be open to debate, and in future repetitions of the process, the team might decide to revise certain classifications. For this reason, the categorization process and its underlying rationale are shared to facilitate reproducibility, as further categorizations for other MITRE matrices may also be required for penetration testing. This is particularly relevant because an ICS attack can

combine techniques originating from different matrices, including ICS, Enterprise, and Mobile matrices.

The methodology focuses on aligning threat modeling outputs with test selection and prioritization; it does not aim to provide a technical, step-by-step exploitation guide for the execution stage of penetration testing. For operational exploitation and post-exploitation procedures, the methodology is designed to be combined with existing technical guides and best-practice resources. Thus, effectiveness claims are limited to planning, prioritization, and traceability rather than exploitation success rates.

This study relies on a single real-world case study and a single penetration testing campaign, which limits the ability to assess robustness, repeatability, and comparative advantages over alternative penetration test planning approaches. The study aims to demonstrate the feasibility and practical value of systematically linking design-phase threat modeling outputs to penetration test planning in an ICS context. The case study, therefore, serves as an exploratory validation, illustrating how the BRIDGE methodology can be applied under realistic industrial constraints. The methodology and its underlying decision rationale are thoroughly documented to ensure transparency and reproducibility. Further validation through additional case studies across different ICS domains and repeated testing campaigns will be required to assess robustness and broader applicability. Nevertheless, the presented case study provides an initial empirical demonstration of how threat modeling outcomes can be systematically incorporated into ICS penetration test planning.

7. Conclusion

This research highlights the value of integrating design-phase threat modeling outcomes into ICS penetration testing plans. To demonstrate how this integration can be achieved, the BRIDGE methodology, a nine-stage threat modeling-driven ICS penetration testing methodology, was proposed and its practical application was demonstrated through a real-world case study. A STRIDE-based categorization of MITRE ATT&CK for ICS techniques was also introduced to support structured test planning. The detected system vulnerability scores were calculated using both CVSS v3.1 and CVSS v4.0. The results indicate that CVSS v4.0 is more effective in capturing the unique risk characteristics of ICS vulnerabilities, particularly those with potential safety or physical consequences. This study contributes a structured approach to ICS penetration testing that improves planning and helps reduce the likelihood of missing critical vulnerabilities. While the findings are promising, further research is needed to develop a comprehensive library of ICS-specific testing procedures. By streamlining test planning and aligning the testing phase with design-phase insights, the proposed methodology supports both practitioners and researchers in enhancing the security and resilience of industrial control systems.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used Google's Gemini and OpenAI's ChatGPT to assist with grammar review and language refinement. No content was generated by these tools; the authors solely authored the content and used AI tools for editorial suggestions. All edits were reviewed and approved by the authors, who take full responsibility for the final version of the manuscript.

CRediT authorship contribution statement

Shaymaa Mamdouh Khalil: Conceptualization, Methodology, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization; **Hayretin Bahsi:** Supervision, Conceptualization, Methodology, Writing - review & editing; **Tarmo Korötko:** Funding acquisition.

Data availability

The data that has been used is confidential.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge *Alvaro Wim Schuller Fernandez* for conducting the penetration testing activities described in this paper, and for the valuable contributions throughout the testing lifecycle. This work was supported by the Estonian Ministry of Education and Research and the [European Regional Development Fund](#) (grant [2014-2020.4.01.20-0289](#)) and by the project "*Increasing the knowledge intensity of Ida-Viru entrepreneurship*" co-funded by the European Union (2021-2027.6.01.23-0034).

Supplementary material

Supplementary material associated with this article can be found in the online version at [10.1016/j.cose.2026.104906](https://doi.org/10.1016/j.cose.2026.104906).

References

- Alexander, O., Belisle, M., Steele, J., 2020. Mitre att&ck for industrial control systems: design and philosophy. The MITRE Corporation: Bedford, MA, USA 29, 21–85.
- Bahsi, H., Dola, H.O., Khalil, S.M., Korötko, T., 2022. A cyber attack taxonomy for microgrid systems. In: 2022 17th Annual System of Systems Engineering Conference (SOSE), pp. 324–331. <https://doi.org/10.1109/SOSE55472.2022.9812642>
- Choi, S., Choi, J., Yun, J.H., Min, B.G., Kim, H.C., 2020. Expansion of ICS testbed for security validation based on MITRE ATT&CK techniques. CSET 2020 - 13th USENIX Workshop on Cyber Security Experimentation and Test, co-located with USENIX Security 2020.
- Commission, I.E., 2019. IEC 62443-4-2 : Security for industrial automation and control systems . Part 4-2 : technical security requirements for IACS components .
- Commission, I. E., 2018. Security for industrial automation and control systems - Part 4-1: secure product development lifecycle requirements.
- Commission, I.E., 2022. IEC 62443-2-1 Industrial communication networks. Network and system security. Part 2-1: establishing an industrial automation and control system security program .
- Da Silva, M., Puys, M., Thevenon, P.H., Mocanu, S., Nkawa, N., 2023. Automated ICS template for STRIDE microsoft threat modeling tool. ACM Int. Conf. Proc. Ser. (January). <https://doi.org/10.1145/3600160.3605068>
- Directive, C., 2008. 114/EC of 8 december 2008 on the identification and designation of european critical infrastructures and the assessment of the need to improve their protection. EUR-Lex.–2008.–23.12, L 345, 75–82.
- DNP Users Group, 2005. A DNP3 Protocol Primer. Technical Report Revision A. DNP Users Group. Accessed: January 2026. <https://www.dnp.org/Portals/0/AboutUs/DNP3%20Primer%20Rev%20A.pdf>.
- FIRST, 2023. Common vulnerability scoring system version 4.0 specification document <https://www.first.org/cvss/>.
- Fujimoto, M., Itani, Y., Mitsunaga, T., 2023. Lessons learned for practical penetration test against industrial control systems. In: 2023 IEEE International Conference on Computing (ICOCO). IEEE, pp. 59–64.
- Howard, M., Lipner, S., 2006. The security development lifecycle. <https://doi.org/10.7765/9781526103482.00008>
- Khalil, S.M., Bahsi, H., Korotko, T., 2026. Penetration testing of newly-deployed industrial control systems. In: Proceedings of the International Conference on Critical Infrastructure Protection. Springer Nature Switzerland, pp. 25–48. First Online: 02 January 2026.
- Khalil, S.M., Bahsi, H., Korötko, T., 2024. Threat modeling of industrial control systems: a systematic literature review. Computers Secur. 136, 103543. <https://doi.org/https://doi.org/10.1016/j.cose.2023.103543>
- Khalil, S.M., Bahsi, H., Ochieng'Dola, H., Korötko, T., McLaughlin, K., Kotkas, V., 2023. Threat modeling of cyber-physical systems-a case study of a microgrid system. Computers & Secur. 124, 102950.
- MITRE Corporation, 2023. Common attack pattern enumeration and classification (CAPEC). <https://capec.mitre.org/>. Accessed: January 2026.
- MITRE Corporation, 2026. Common weakness enumeration (CWE). <https://cwe.mitre.org/>. Accessed: January 2026.
- Modbus Organization, 2012. MODBUS Application Protocol Specification V1.1b3. Technical Report. Modbus Organization. Accessed: January 2026. https://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf.

- OWASP, . Wstg - latest | owasp foundation. 2026 <https://owasp.org/www-project-web-security-testing-guide/latest/>.
- OWASP, 2023. Owasp internet of things project. https://wiki.owasp.org/index.php/OWASP_Internet_of_Things_Project#tab=IoT_Top_10. Accessed: 2023-04-30.
- Penetration Testing Execution Standard, . Ptes technical guidelines. 2026 <http://www.pentest-standard.org/>.
- Rapid7, 2026. Metasploit framework. <https://www.metasploit.com/>. Accessed: January 2026.
- Saad, E., Mitchell, R., 2020. OWASP Web security testing guide version 4.2. The OWASP Foundation , 465.
- Salzillo, G., Rak, M., Moretta, F., 2020. Threat modeling based penetration testing: the open energy monitor case study. ACM International Conference Proceeding Series . <https://doi.org/10.1145/3433174.3433181>
- Searle, J., Rasche, G., Wright, A., Dinnage, S., 2016. NESCOR guide to penetration testing for electric utilities. National Electric Sector Cybersecurity Organization Resource (NESCOR).
- Security for Industrial Automation and Control Systems - Part 2-3: Patch Management in the IACS Environment, 2015. IEC Technical Report IEC TR 62443-2-3. International Electrotechnical Commission (IEC).
- Sharma, A., Sabharwal, S., Nagpal, S., 2023. A hybrid scoring system for prioritization of software vulnerabilities. Computers Secur. 129. <https://doi.org/10.1016/j.cose.2023.103256>
- Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., Thompson, M., 2023. Guide to Operational Technology (OT) Security. Technical Report NIST SP 800-82 Rev. 3. National Institute of Standards and Technology, Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-82r3>
- Strom, B.E., Applebaum, A., Miller, D.P., Nickels, K.C., Pennington, A.G., Thomas, C.B., 2018. Mitre att&ck: Design and Philosophy. In: Technical Report. The MITRE Corporation.
- Süren, E., Fredrik, H., Olegård, J., Lagerström, R., 2023. PatIoT: practical and agile threat research for IoT. Int. J. Inf. Secur. 22, 213–233. <https://doi.org/10.1007/s10207-022-00633-3>
- Tharot, K., Duong, Q.B., Riel, A., Thiriet, J.M., 2023. Industrial cybersecurity game-scenarios based on the MITRE ATT&CK framework. Conference Proceedings - 2023 IEEE Asia Meeting on Environment and Electrical Engineering, EEE-AM 2023 , 1–4. <https://doi.org/10.1109/EEE-AM58328.2023.10395155>
- ZigBee Alliance, 2015. ZigBee Specification. Technical Report Document 05-3474-21. ZigBee Alliance. Accessed: January 2026. <https://zigbeealliance.org/wp-content/uploads/2019/11/docs-05-3474-21-0csg-zigbee-specification.pdf>